

Querying Data and Automating Data Ingestion in HydroServer

Sara Alonso Vicario & Emily Dubois

Applied Scientist & UI/UX Developer, Center for Geospatial Solutions

Session 7a. Querying Data and Automated Data Workflows in HydroServer

Tuesday, September 15, 2026

15:35 – 17:30



Utah Water Research Laboratory
UtahStateUniversity



This workshop is supported with funding from the United States government.



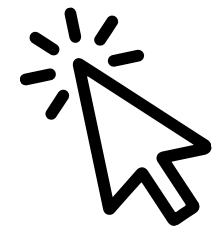
What We've Done So Far

In the previous session, you:

- Automated data ingestion using the Streaming Data Loader
- Performed data quality analysis

Now, you're ready to learn how to **retrieve the data you've uploaded to HydroServer** and **explore other ways to automate observations loading!**

Please go to:



and take a couple of minutes to read through what we're going to do in this **Session 7a** • ~ •

Session 7a - Querying Data and Automated Data Workflows in HydroServer

Wednesday, September 16, 2026 | 15:35 – 17:30

Session Overview

In this session, you will learn how to retrieve data from HydroServer using the [hydroserverpy](#) package and explore additional methods for automating data ingestion workflows. You will work through two practical exercises.

What You Will Do

Exercise 3: Querying Data from HydroServer

During the third exercise, you will learn how to query data stored in HydroServer. In particular, you will retrieve the real-time stage data that you uploaded using the [Streaming Data Loader](#) and quality-controlled using the [HydroServer QC App](#) in Session 6a.

Using Python and the [hydroserverpy](#) package, you will:

- Retrieve information about your workspace, monitoring sites, datastreams, and observations.
- Access and explore your quality-controlled observations programmatically.
- Explore additional approaches for automating data ingestion workflows.

Follow the presentation

We will go through this [Presentation](#) together step by step. You can refer back to the presentations after the training to review key concepts and exercise steps.

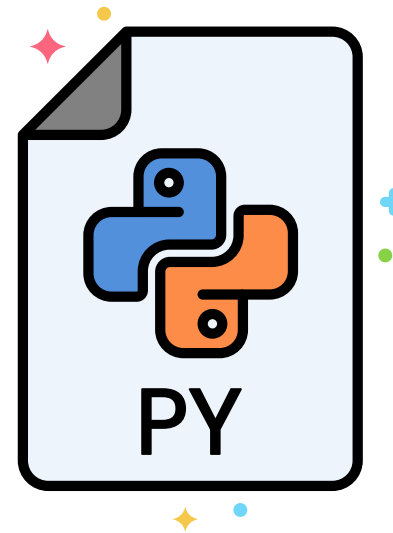
Now, we are going to retrieve data from HydroServer by completing **Exercise 3**.



Exercise 3:

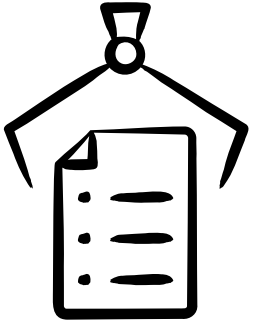
**Query Data and Metadata from
HydroServer using hydrosverpy**

In this exercise, we will query the data using the **hydroserverpy** package, which is designed to interact with HydroServer.



This is the same package we used in the previous exercises, but this time we will use it to **retrieve data** instead of ingesting it! 🙌

In this exercise, we will query:



- **HydroServer resources**, including workspaces and monitoring sites.
- **Metadata** from monitoring sites and datastreams.
- **Observations** from the Kanzenze station in Rwanda that were uploaded using the Streaming Data Loader and quality-controlled using the HydroServer QC App (see Session 6a).



Access Exercise 3

The third exercise can be accessed through the following link for each country:



Kenya: [Access Exercise 3](#)



Rwanda: [Access Exercise 3](#)



Uganda: [Access Exercise 3](#)



Ethiopia: [Access Exercise 3](#)

You may be asked to sign in to your Google account.





Commands + Code + Text ▶ Run all Copy to Drive

Connect ^

Install hydroserverpy

For this workshop, we will use Google Colab to run the exercises. Before starting, run the code cell below to install the required version of the hydroserverpy package. The current version of hydroserverpy used for this training is [1.11.3](#).

```
[ ] !pip install hydroserverpy==1.11.3
```

> ... [Show hidden output](#)

Import Required Packages

The following packages and modules

- **hydroserverpy** – Connects to
- **pandas** – Reads, organizes, and
- **datetime** – Works with dates and
- **getpass** – Allows you to enter your HydroServer password securely without displaying it on the screen.

Warning: This notebook was not authored by Google

This notebook is being loaded from [GitHub](#). It may request access to your data stored with Google, or read data and credentials from other sessions. Please review the source code before executing this notebook.

Cancel

[Run anyway](#)

**If you see this message
when opening the
Google Colab notebook,
click “Run anyway.”**

```
[ ] # Import HydroServer to connect to and manage HydroServer resources
    from hydroserverpy import HydroServer

    # Import pandas to read, organize, and process the sensor data
    import pandas as pd

    # Import datetime to work with dates and times
    from datetime import datetime
```

REMEMBER



Do not start running
the code it yet

If you see this message while running the code, click "Reconnect." Do not rerun the cells you already completed, as this may create duplicate resources.

Runtime disconnected

Your runtime has been disconnected due to inactivity or reaching its maximum duration. [Learn more](#)

If you are interested in longer runtimes with more lenient timeouts, you may want to check out [Colab Pro](#).

Close

[Reconnect](#)

September

will use this

1. Getting Started

Install hydrosolverpy

For this workshop, we will use Google Colab to run the exercises. Before starting, run the code cell below to install the required version of the hydrosolverpy package. For this exercise, we will connect to the HydroServer Playground instance at playground.hydroserver.org. The current version of hydrosolverpy used for this training is [1.11.3](#).

```
[ ] !pip install hydrosolverpy==1.11.3
```

**DON'T
FORGET**

Getting Started with HydroServer Programmatically

Python code

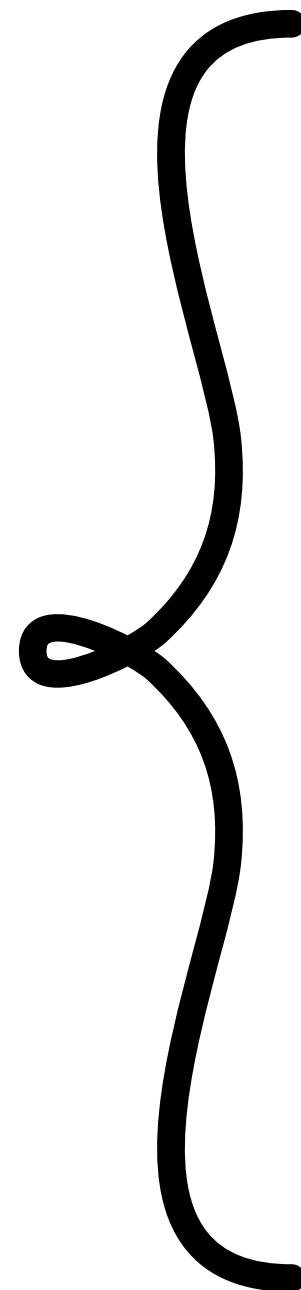
REMINDER

Install
hydroserverpy
package



```
!pip install hydroserverpy==1.11.3
```

Import the **Python packages** needed for the exercise



```
# Import HydroServer to connect to and manage HydroServer resources  
from hydroserverpy import HydroServer
```

```
# Import pandas to read, organize, and process the sensor data  
import pandas as pd
```

```
# Import datetime to work with dates and times  
from datetime import datetime
```

```
# Import getpass to securely enter your HydroServer password  
from getpass import getpass
```

These steps are the same as in the previous exercises.

Getting Started with HydroServer Programmatically

```
Collecting dnspython>=2.0.0 (from email-validator>=2.0.0->pydantic[email]>=2.0->hydroserverpy==1.11.3)
  Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Downloading hydroserverpy-1.11.3-py3-none-any.whl (113 kB)
  113.4/113.4 kB 5.6 MB/s eta 0:00:00
Downloading croniter-6.2.4-py3-none-any.whl (46 kB)
  46.7/46.7 kB 3.3 MB/s eta 0:00:00
Downloading jmespath-1.1.0-py3-none-any.whl (20 kB)
Downloading python_crontab-3.4.0-py3-none-any.whl (27 kB)
Downloading email_validator-2.3.0-py3-none-any.whl (35 kB)
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
  331.1/331.1 kB 18.2 MB/s eta 0:00:00
Installing collected packages: python-crontab, jmespath, dnspython, email-validator, croniter, hydroserverpy
Successfully installed croniter-6.2.4 dnspython-2.8.0 email-validator-2.3.0 hydroserverpy-1.11.3 jmespath-1.1.0 python-crontab-3.4.0
```



When the packages have been successfully installed and imported, you will see these messages!

[4]
✓ 0s

```
# Import HydroServer to connect to and manage HydroServer resources
from hydroserverpy import HydroServer

# Import pandas to read, organize, and process the sensor data
import pandas as pd

# Import datetime to work with dates and times
from datetime import datetime

# Import getpass to securely enter your HydroServer password
from getpass import getpass
```



Programmatically – Set the Initial Parameters for Login

Set the initial parameters to connect:

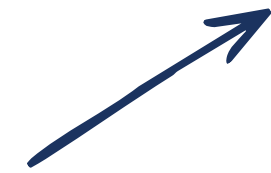
- Host
- E-mail
- Password



```
# Set initial parameters to connect to HydroServer
```

```
hydroserver_host = 'https://playground.hydroserver.org'
```

HydroServer Playground URL



```
# Change the email and password below to your HydroServer username and password
```

```
hydroserver_email = 'svicario@lincolninst.edu'
```

```
hydroserver_password = getpass('Enter your HydroServer password: ')
```

IMPORTANT!

When you run the code, you will be prompted to enter your password.



```
# Set initial parameters to connect to HydroServer
hydroserver_host = 'https://playground.hydroserver.org'

# Change the email and password below to your HydroServer username and password
hydroserver_email = 'svicario@lincolninst.edu' #'user@youremail.com'
hydroserver_password = getpass('Enter your HydroServer password: ') #getpass('Enter your HydroServer password: ')
```

Enter your HydroServer password:



These steps are the same as in the previous exercises.

Reminder

Programmatically – Log in into HydroServer

Reminder

The next step is to **establish a connection with HydroServer** using the parameters you just set up.

```
# Initialize HydroServer connection with credentials.
hs = HydroServer(
    host='https://playground.hydroserver.org',
    email='your@email.com',
    password='yourpassword'
)
print('\nSuccessfully connected to HydroServer!')
```

```
# Initialize HydroServer connection with credentials.
hs = HydroServer(
    host=hydroserver_host,
    email=hydroserver_email,
    password=hydroserver_password
)

print('\nSuccessfully connected to HydroServer!')
```

...

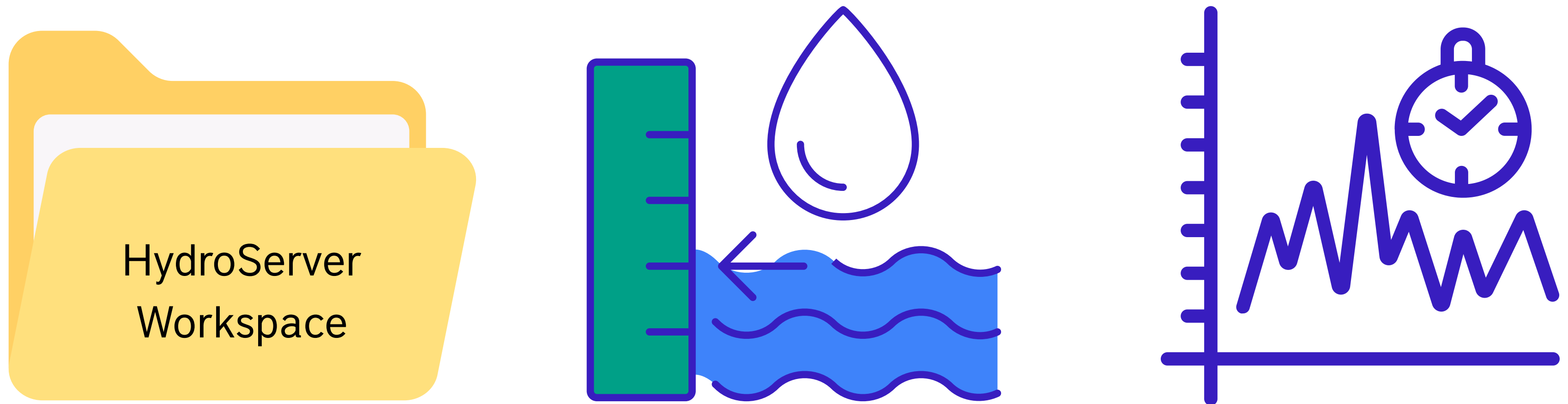
Successfully connected to HydroServer!



If the connection is successful, you will see this message in your console

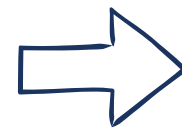
These steps are the same as in the previous exercises.

Next, let's start querying HydroServer resources!



QUERY WORKSPACES

List of all
workspaces you
have access to
(including public)



```
workspaces = hs.workspaces.list()
```

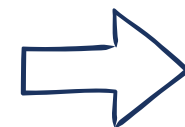
```
# Print the names of the first 5 workspaces that were returned:
```

```
print(f'\nList of workspaces I have access to:\n')
```

```
for workspace in workspaces.items[:5]:
```

```
    print(workspace.name)
```

Expected Output:
List of the first 5
Public Workspaces



```
List of workspaces I have access to:
```

```
Test Workspace
```

```
Test Workspace #2
```

```
Stream gauge register
```

```
Vincent
```

```
DRE
```



**Public
Workspace
Names**

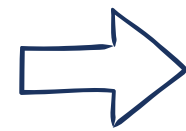


QUERY WORKSPACES YOU ARE AFFILIATED WITH

Workspaces you own and
workspaces you have access to



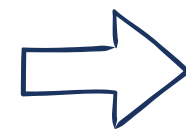
List of all
workspaces you you
either own them or
you have been
given access



```
my_workspaces = hs.workspaces.list(is_associated=True)
```

```
print(f'\nList of workspaces I am affiliated with:\n')  
for workspace in my_workspaces.items:  
    print(workspace.name)
```

List of
workspaces I
am affiliated
with



```
List of workspaces I am affiliated with:  
Rwanda Training 2026 - Sara
```





Transfer workspace ownership

Submit

? You can also do it programmatically

```
# Get the workspace
workspace = hs.workspaces.get(workspace_id)

# Display available roles and their IDs
for role in workspace.roles:
    print(role.name, role.uid)

# Get the Editor role
editor_role = next(
    role for role in workspace.roles
    if role.name == "Editor"
)

# Add a collaborator
workspace.add_collaborator(
    email="collaborator@email.com",
    role=editor_role
)
```

You can add collaborators (give access) to your workspaces through the **Web Dashboard.**

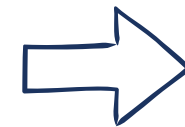
REMEMBER!

QUERY A WORKSPACE WITH A SPECIFIC NAME

Change for your
workspace name



Find your
specific
workspace by
searching for its
name.



```
workspace = [  
    workspace  
    for workspace in my_workspaces.items  
    if workspace.name == "Rwanda Training 2026 - Sara" # Change this name to your  
workspace name  
][0]  
  
workspace_id = workspace.uid  
  
print(f"\nRetrieved workspace: {workspace.name}")  
print(f"\nRetrieved workspace uid: {workspace_id}")
```

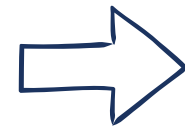
```
Retrieved workspace: Rwanda Training 2026 - Sara
```

```
Retrieved workspace uid: 01a062cb-7ae3-7dd6-a6d6-0ea739c93849
```



QUERY MONITORING SITES (THINGS) IN MY WORKSPACE

Get all the
monitoring
sites in the
workspace



```
# Limit the list to the Things in a particular workspace
```

```
workspace_things = workspace.things
```

```
for thing in workspace_things:
```

```
    print(thing.name)
```

Kanzenze Hydrological Station

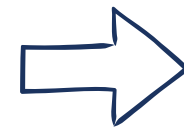


**If you have more than one monitoring
site in your workspace, you will see
all their names here.**



QUERY A MONITORING SITE WITH A SPECIFIC NAME

Get a
monitoring site
using its name



```
thing = [  
    thing  
    for thing in workspace_things  
    if thing.name == "Kanzenze Hydrological Station" # Set the name of  
the thing you want to find  
][0]  
  
thing_id = thing.uid # Get the ID for a thing  
  
print(f"\nRetrieved thing: {thing.name}")
```

Filtering for the Kanzenze
Hydrological Station

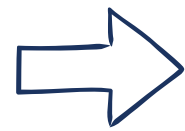


Retrieved thing: Kanzenze Hydrological Station



QUERY ATTRIBUTES OF A MONITORING SITE (THING)

Get the
attributes of the
monitoring site



```
# Get attributes of a Thing
print(f'\nExtracting attributes of a Thing:\n')
print(f'Site name: {thing.name}')
print(f'Site description: {thing.description}')
print(f'Latitude: {thing.latitude}')
print(f'Longitude: {thing.longitude}')
print(f'Site type: {thing.site_type}')
```



These are the attributes we added when we created the monitoring site in Exercises 1 and 2, based on the data model you designed during the interactive exercise in the first session.

Very intuitive!

- **thing.name**
- **thing.latitude**
- **thing.longitude**
- ...



Extracting attributes of a Thing:

```
Site name: Kanzenze Hydrological Station
Site description: Hydrological monitoring station on the Nyabarongo River at Kanzenze, Rwanda.
Latitude: -2.0613
Longitude: 30.0877
Site type: Stream
```



ADDITIONAL EXAMPLES OF QUERYING MONITORING SITE (THING) METADATA

This is the list
of attributes
you can query
from the
monitoring
site metadata
using Python.

Python

```
1  # Basic information
2  print(f"UID: {thing.uid}")
3  print(f"Name: {thing.name}")
4  print(f"Description: {thing.description}")
5
6  # Monitoring site information
7  print(f"Sampling feature type: {thing.sampling_feature_type}")
8  print(f"Sampling feature code: {thing.sampling_feature_code}")
9  print(f"Site type: {thing.site_type}")
10
11 # Location
12 print(f"Latitude: {thing.latitude}")
13 print(f"Longitude: {thing.longitude}")
14 print(f"Elevation: {thing.elevation_m}")
15 print(f"Elevation datum: {thing.elevation_datum}")
16
17 # Administrative location
18 print(f"Administrative area 1: {thing.admin_area_1}")
19 print(f"Administrative area 2: {thing.admin_area_2}")
20 print(f"Country: {thing.country}")
21
22 # Data information
23 print(f"Data disclaimer: {thing.data_disclaimer}")
24
25 # Workspace
26 print(f"Workspace ID: {thing.workspace_id}")
27 print(f"Workspace name: {thing.workspace.name}")
28
29 # Visibility
30 print(f"Private: {thing.is_private}")
31
32 # Tags
33 print(f"Tags: {thing.tags}")
34
35 # Datastreams
36 print(f"Datastreams: {thing.datastreams}")
```

Basic information

Unique identifier, name, and description of the monitoring site.

Monitoring site information

Details about the sampling feature and site type.

Location

Geographic coordinates and elevation details.

Administrative location

Political subdivision and country where the site is located.

Data information

Disclaimer or notes about the data associated with the site.

Workspace

Information about the workspace that contains this site.

Visibility

Indicates whether the site is private (True/False).

Tags

User-defined keywords or labels for the site.

Datastreams

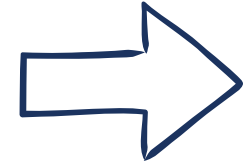
List of datastream objects associated with this site.



You can run these
queries in the code to
explore all the
available attributes!

QUERY DATASTREAMS FROM A MONITORING SITE (THING)

Get the
Datastreams
Contained in the
Monitoring Site



```
# Get the Datastreams for the selected monitoring site (Thing)
```

```
thing_datastreams = thing.datastreams
```

```
# Loop through the Datastreams associated with the monitoring site
```

```
for datastream in thing_datastreams:
```

```
    print(datastream.name)
```

```
Datastream ID: 01a066bf-f2e7-770e-8d87-8a780e8fb344
```

```
Datastream name: Stage - Real-time - Kanzenze Hydrological Station
```

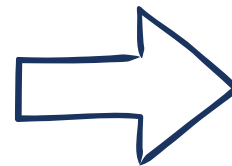


QUERY A DATASTREAM WITH A SPECIFIC NAME

**Filtering for the datastream
containing real-time data
from the Kanzenze station.**



Get the name and
UID of a
datastreams by its
name



```
# Name of the datastream
datastream_name = "Stage - Real-time - Kanzenze Hydrological Station"

# Get the datastream by name
ds = next(
    ds for ds in thing.datastreams
    if ds.name == datastream_name
)

# Get its identifier
ds_id = ds.uid

print("Datastream ID:", ds_id)
print("Datastream name:", ds.name)
```

```
Datastream ID: 01a066bf-f2e7-770e-8d87-8a780e8fb344
Datastream name: Stage - Real-time - Kanzenze Hydrological Station
```



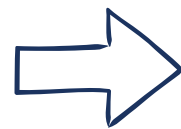
QUERY ATTRIBUTES OF A DATASTREAM

Very intuitive!



- **ds.name**
- **ds.observed_property.name**
- **ds.processing_level.definition**
- ...

Query the
metadata of
that
datastream



```
# Retrieve a datastream object containing the metadata for the datastream
print(f"\nRetrieving information for datastream {ds_id}\n")
print(f"Datastream name: {ds.name}")
print(f"Observed property name: {ds.observed_property.name}")
print(f"Processing level: {ds.processing_level.definition}")
print(f>Description: {ds.description}")
print(f"Unit: {ds.unit.name}")
print(f"Unit abbreviation: {ds.unit.symbol}")
```

```
Datastream name: Stage - Real-time - Kanzenze Hydrological Station
Observed property name: Stage
Processing level: Raw Data
Description: Real-time stage observations at Kanzenze Hydrological Station
```



ADDITIONAL EXAMPLES OF QUERYING DATASTREAM METADATA

This is the list of attributes you can query from the datastream metadata using Python.

Python

```
1 # Basic information
2 print(f"UID: {ds.uid}")
3 print(f"Name: {ds.name}")
4 print(f"Description: {ds.description}")
5
6 # Observed property
7 print(f"Observed property name: {ds.observed_property.name}")
8 print(f"Observed property definition: {ds.observed_property.definition}")
9 print(f"Observed property description: {ds.observed_property.description}")
10 print(f"Observed property code: {ds.observed_property.code}")
11
12 # Processing level
13 print(f"Processing level code: {ds.processing_level.code}")
14 print(f"Processing level definition: {ds.processing_level.definition}")
15 print(f"Processing level explanation: {ds.processing_level.explanation}")
16
17 # Unit
18 print(f"Unit name: {ds.unit.name}")
19 print(f"Unit symbol: {ds.unit.symbol}")
20 print(f"Unit definition: {ds.unit.definition}")
21 print(f"Unit type: {ds.unit.unit_type}")
22
23 # Sensor
24 print(f"Sensor name: {ds.sensor.name}")
25 print(f"Sensor description: {ds.sensor.description}")
26
27 # Monitoring site
28 print(f"Monitoring site: {ds.thing.name}")
29
30 # Observation metadata
31 print(f"Observation type: {ds.observation_type}")
32 print(f"Sampled medium: {ds.sampled_medium}")
33 print(f"No-data value: {ds.no_data_value}")
34 print(f"Aggregation statistic: {ds.aggregation_statistic}")
35
36 # Time information
37 print(f"Time aggregation interval: {ds.time_aggregation_interval}")
38 print(f"Time aggregation interval unit: {ds.time_aggregation_interval_unit}")
39 print(f"Intended time spacing: {ds.intended_time_spacing}")
40 print(f"Intended time spacing unit: {ds.intended_time_spacing_unit}")
41
42 # Status
43 print(f"Status: {ds.status}")
44 print(f"Result type: {ds.result_type}")
45 print(f"Value count: {ds.value_count}")
46
47 # Observation period
48 print(f"Phenomenon begin time: {ds.phenomenon_begin_time}")
49 print(f"Phenomenon end time: {ds.phenomenon_end_time}")
50
51 # Result period
52 print(f"Result begin time: {ds.result_begin_time}")
53 print(f"Result end time: {ds.result_end_time}")
54
55 # Visibility
56 print(f"Private: {ds.is_private}")
57 print(f"Visible: {ds.is_visible}")
```

Basic information

Unique identifier, name, and description of the datastream.

Observed property

Information about the variable that is being observed.

Processing level

Indicates the level of processing applied to the observations.

Unit

The units in which the observations are reported.

Sensor

The instrument or device used to collect the data.

Monitoring site

The monitoring location (Thing) where the data is collected.

Observation metadata

Additional metadata about the observations in the datastream.

Time information

Information about the time aggregation and intended sampling interval.

Status

Current status of the datastream, result type, and number of values available.

Observation period

The period over which the phenomenon (real-world event) is observed.

Result period

The period over which the results (data) are available.

Visibility

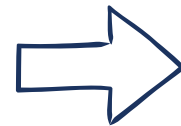
Indicates whether the datastream is private and/or visible.



You can run these queries in the code to explore all the available attributes!

QUERY OBSERVATIONS FROM A DATASTREAM

**Get all the
observations
from the
datastream**



```
# This function call will be limited to 100,000 Observations  
# obs_df = ds.get_observations().dataframe
```

```
# This function call will retrieve all Observations for the Datastream  
# obs_df = ds.get_observations(fetch_all=True).dataframe
```

```
# This function call will get the Observations between two timestamps  
obs_df = ds.get_observations(  
    phenomenon_time_min=datetime(year=2025, month=12, day=31),  
    phenomenon_time_max=datetime(year=2026, month=8, day=13)  
).dataframe
```

```
print(f'\nRetrieved {len(obs_df)} Observations for {ds.name} datastream')
```

Retrieved 32205 Observations for Stage - Real-time - Kanzenze Hydrological Station datastream

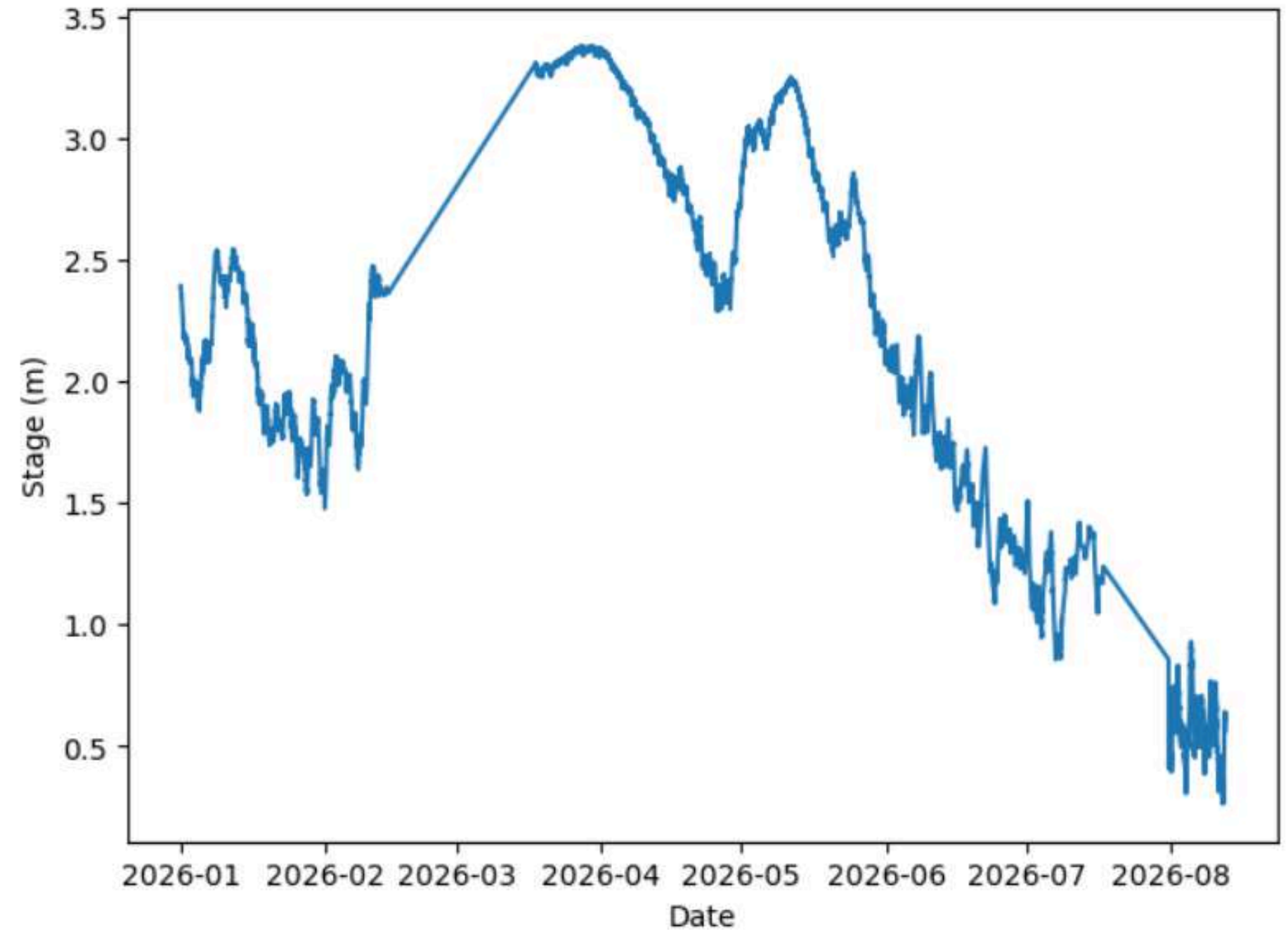
**We have successfully retrieved the quality-
controlled (QC) data from Kanzenze!**

SUCCESS



MAKE A PLOT OF THE OBSERVATIONS

```
# Make a quick plot of the Datastream using matplotlib
plt.plot(obs_df['phenomenon_time'], obs_df['result'],
         linestyle='-')
plt.xlabel('Date')
# Use the observed property name and unit from the
# Datastream metadata to label the y-axis
plt.ylabel(f'{ds.observed_property.name}
           ({ds.unit.symbol})')
plt.tight_layout()
plt.show()
```



The observations retrieved and plotted here are the same ones we previously visualized in the Visualize Data tab of the HydroServer Dashboard.



You have learned how to query resources from
HydroServer. Congratulations!



Now, let's move on to the exercise.

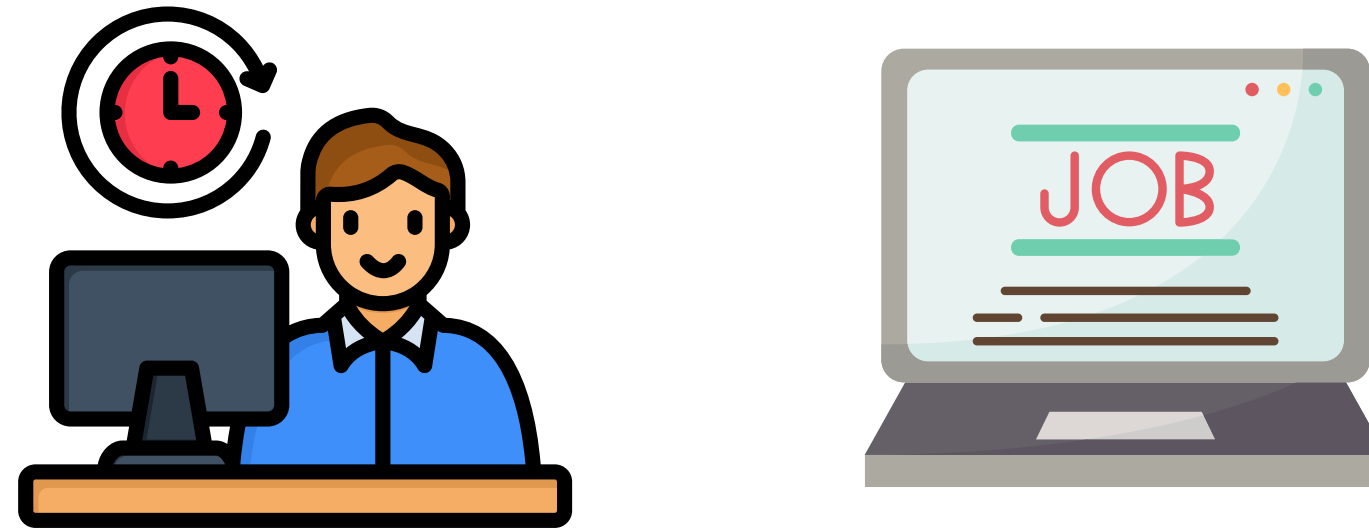
Now, we are going to automate data ingestion into HydroServer using Python by completing **Exercise 4**.



Exercise 4:

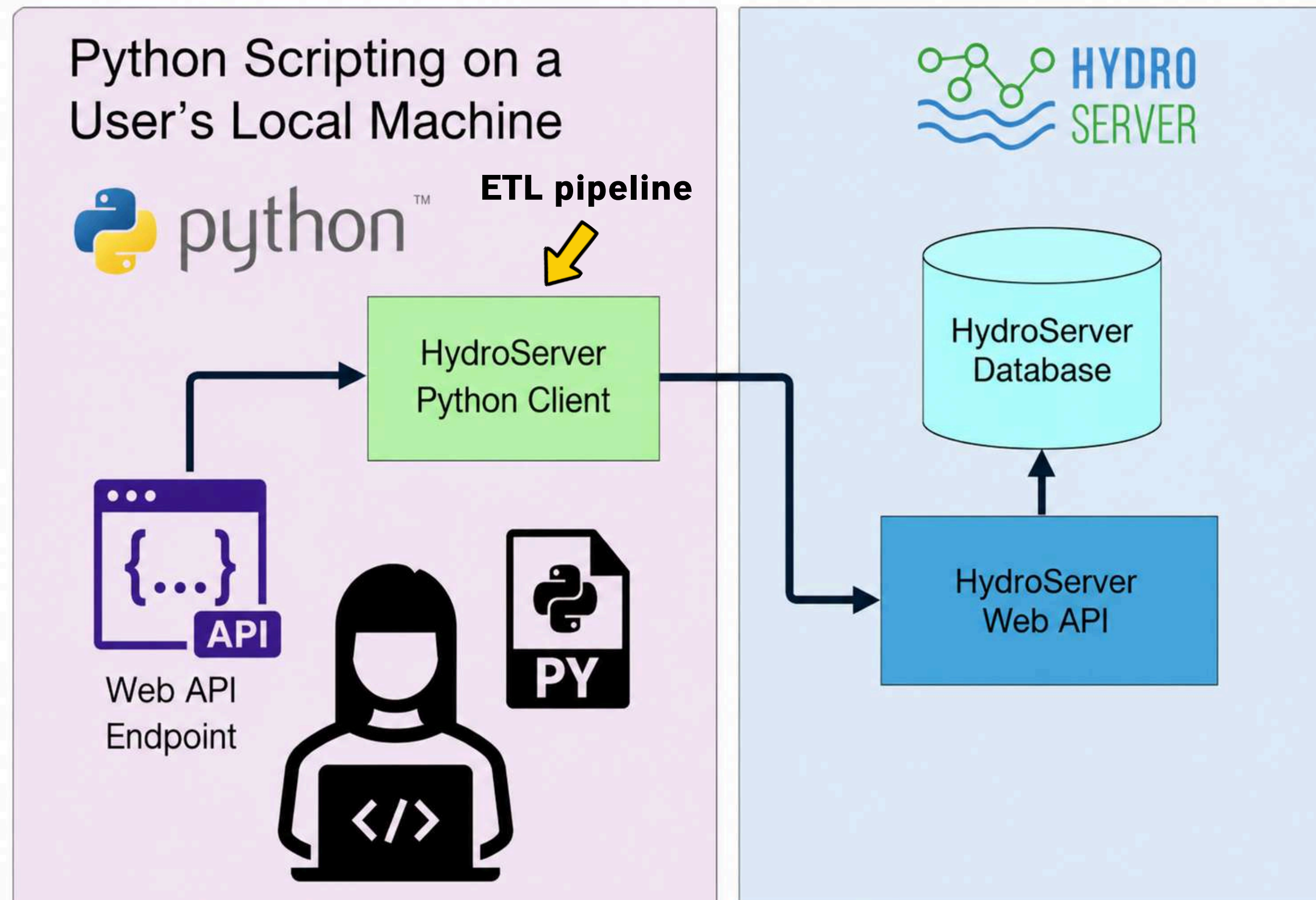
**Automating Data Ingestion into
HydroServer with Externally Managed
Extract-Transform-Load (ETL) Tasks**

In the previous session, you learned how to ingest data into HydroServer using the Streaming Data Loader App from CSV files stored on your computer or server.



In this session, you will learn how to retrieve data from a **Web API** and **schedule a job** to automate data ingestion into HydroServer.

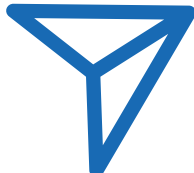
In this automated data ingestion workflow, data are retrieved directly from an external Web API using Python, processed or transformed as needed, and then uploaded to HydroServer using the HydroServer Python Client (hydroserverpy), which sends the observations to HydroServer through the API.



Automated Data Ingestion: How It Works

EXTRACT 1. Python retrieves data from the external Web API. 

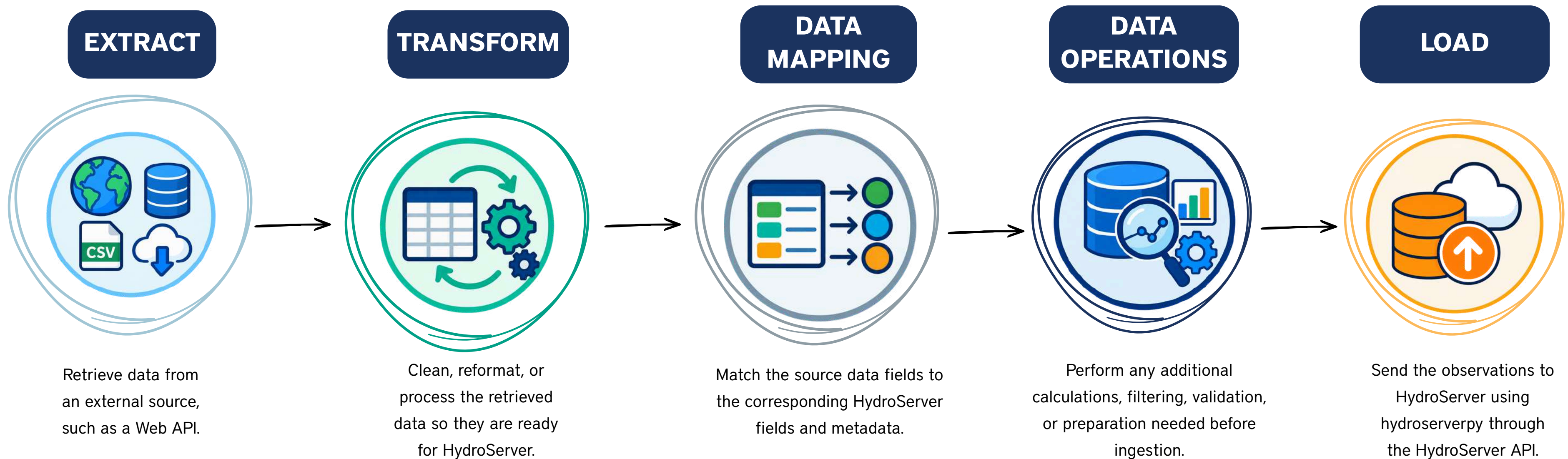
TRANSFORM 2. Python cleans and transforms the data as needed. 

LOAD 3. hydrosverpy sends the prepared data to HydroServer. 

LOAD 4. HydroServer validates and stores the data. 

Extract-Transform-Load (ETL) Pipeline

This ETL pipeline shows how data move from the source to HydroServer.
We will automate these steps using Python and hydroserverpy.



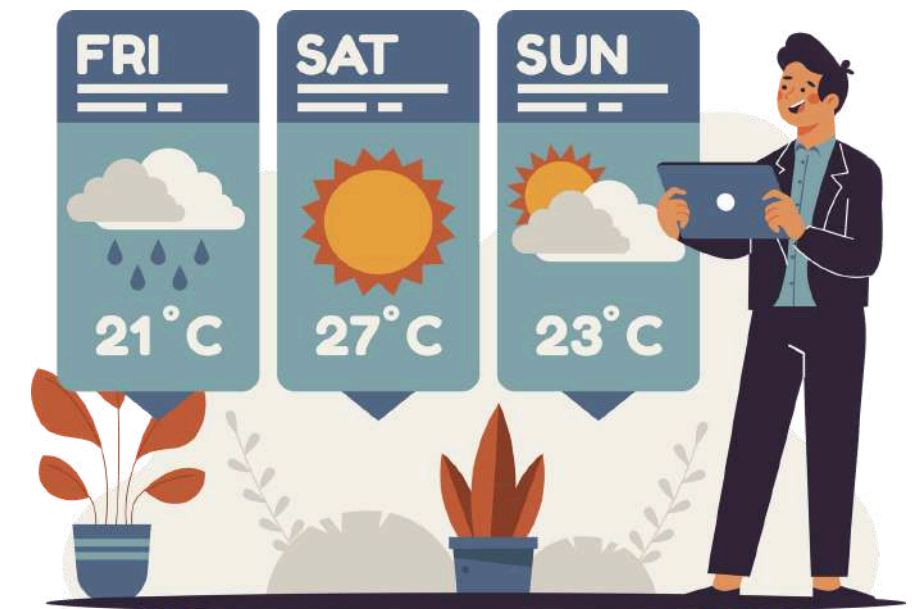
In this exercise, we will retrieve **GEOGLOWS streamflow forecasts** for a station on the **Nzoia River in Kenya** through an **API** and **load** the observations into **HydroServer**.



We will also demonstrate how this data ingestion workflow can be automated. A similar workflow can be used to ingest data from other Web APIs.

What is GEOGLOWS?

- **Global hydrological modeling and forecasting initiative** that provides streamflow information and forecasts for rivers around the world.
- Uses ECMWF* weather forecasts to generate streamflow forecasts.
- Identifies each river reach with a unique River ID.
- **Provides data through the GEOGLOWS API.**



*ECMWF: European Centre for Medium-Range Weather Forecasts.

If you want to explore the streamflow forecasts, you can access them here:
[GEOGLOWS streamflow forecast](#)



GEOGLOWS ECMWF Streamflow REST API Service

This website contains information about the GEOGLOWS RFS and hosts a REST data service for accessing the model results. Click on the map below to begin exploring forecasted streamflow around the world or use the [GEOGLOWS River Forecast System web app](#). For more information about how to access GEOGLOWS data, please visit training.geogloss.org.

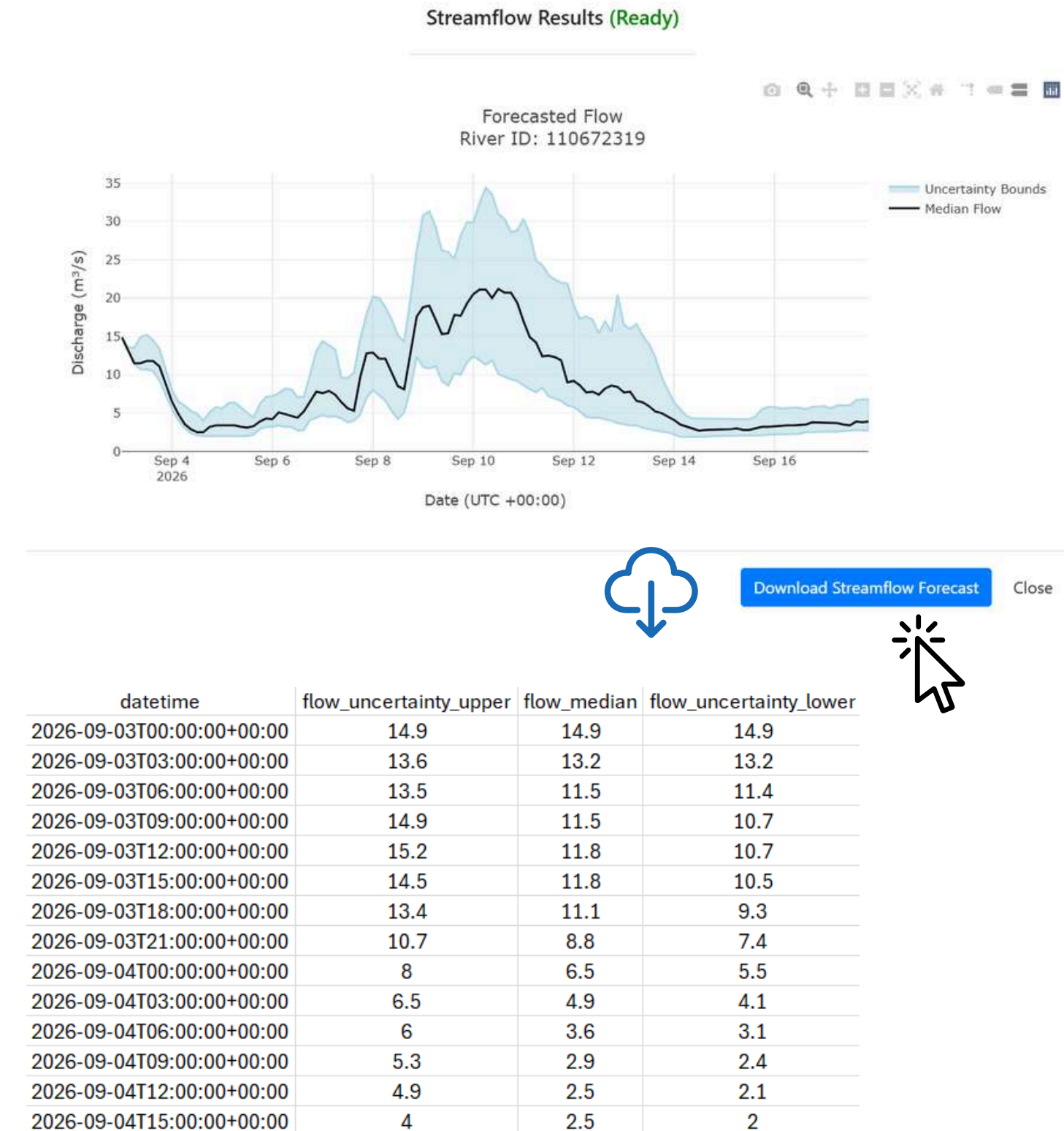


GEOGLOWS STREAMFLOW FORECASTS



The main characteristics of the streamflow forecasts are:

- **Updated: Daily**
- **Forecast horizon: 15 days**
- **Temporal resolution: 3-hourly average streamflow**
- **Ensemble: 51 members**

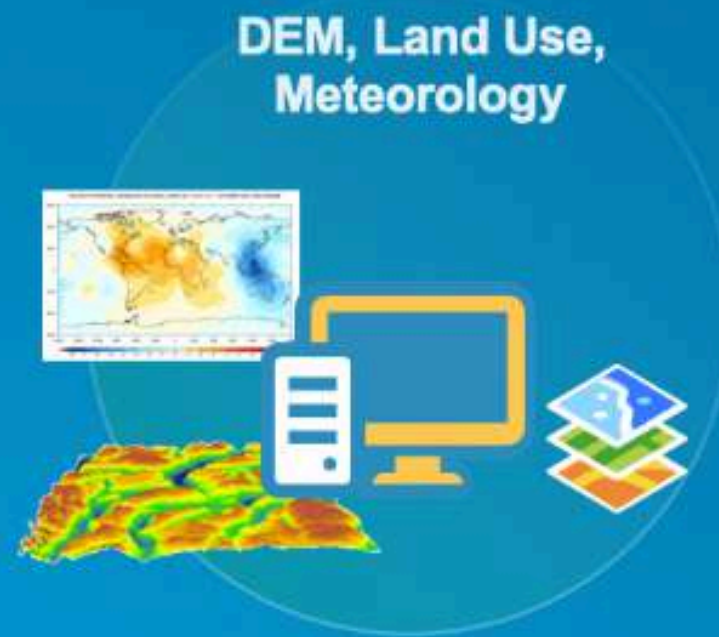


Example of a GEOGLOWS streamflow forecast for River ID 110672319.

To learn more, visit the GEOGLOWS ECMWF website.



From This



Global
Data



Hydrologic Modeling



HydroMet Services



Local Governments



NGO's and GEO Partners

To This



Hydrologic Modeling



Global
Services



HydroMet Services



Local
Governments

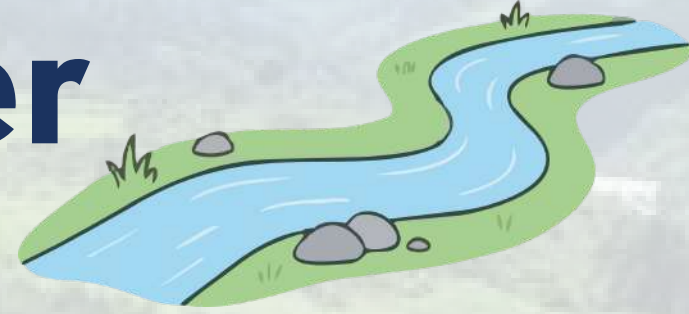


NGO's and
GEO Partners

A stylized illustration of a winding river in shades of blue, flowing from the top right towards the bottom left. The river has several meanders and is surrounded by light blue, rounded shapes representing rocks or islands. The background is a solid light blue.

Example: Nzoia River in Kenya

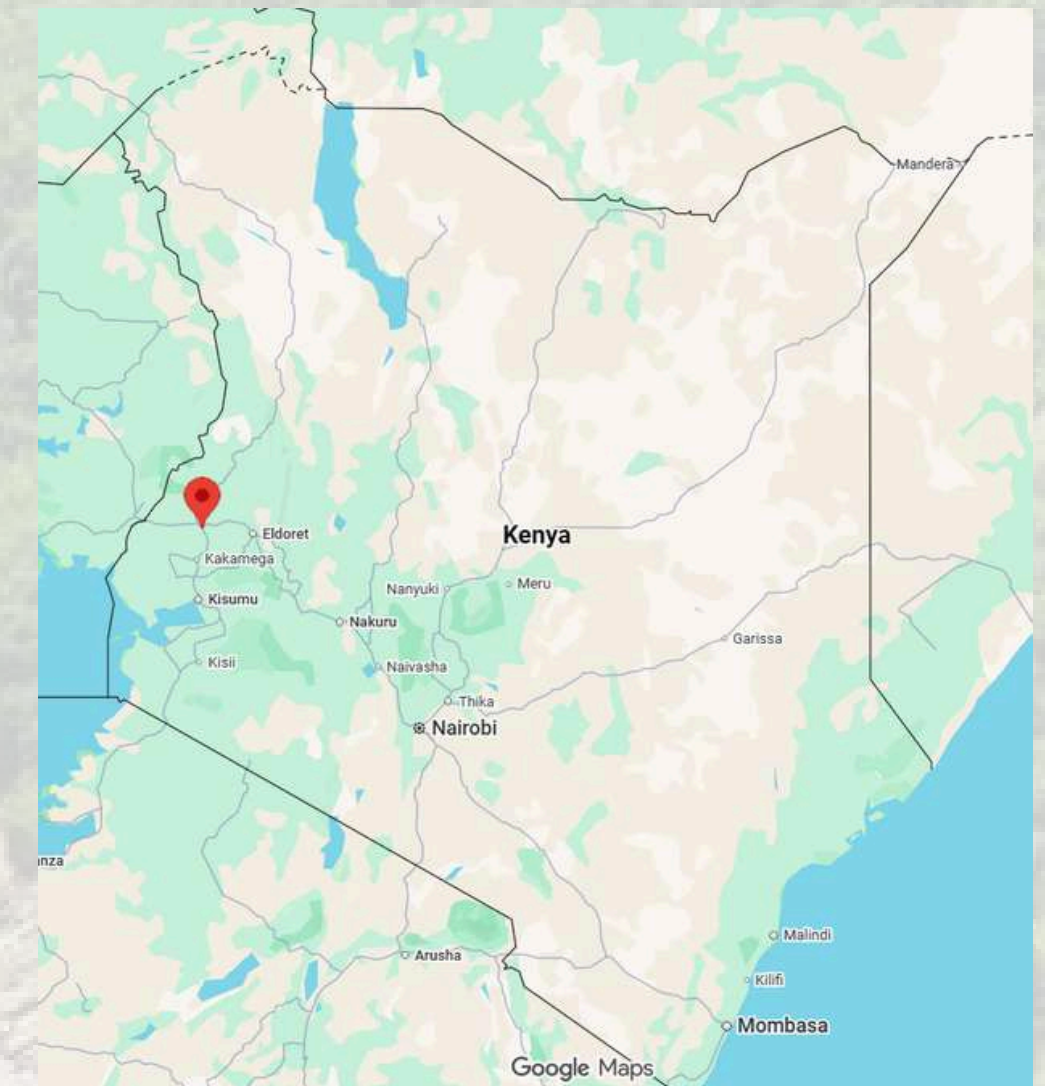
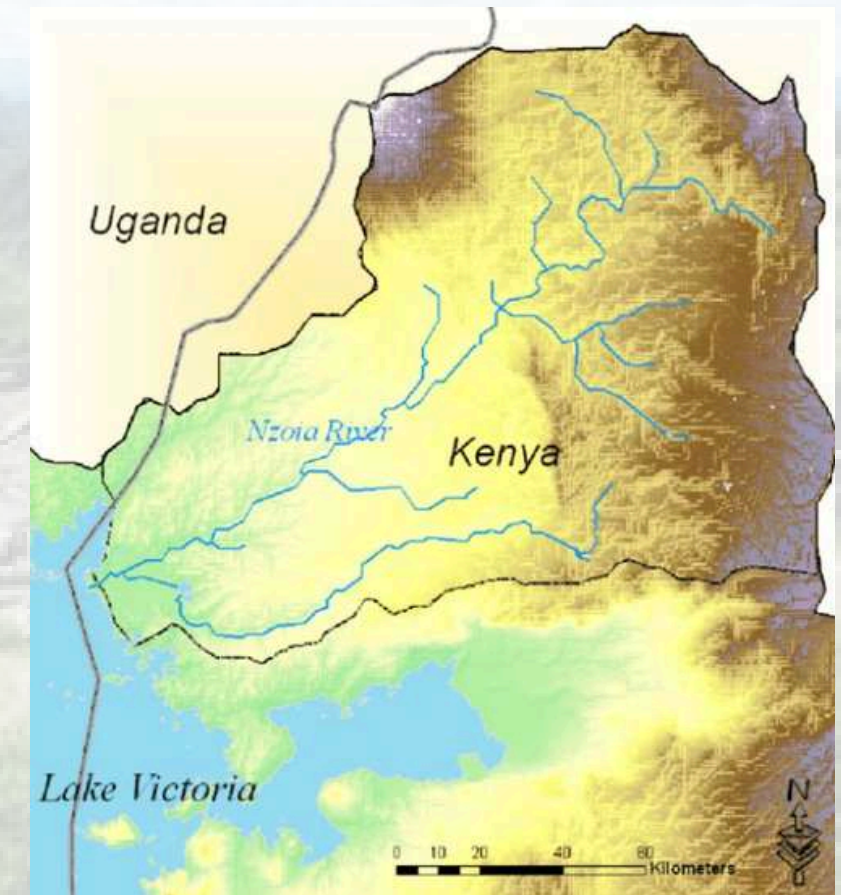
Nzoia River



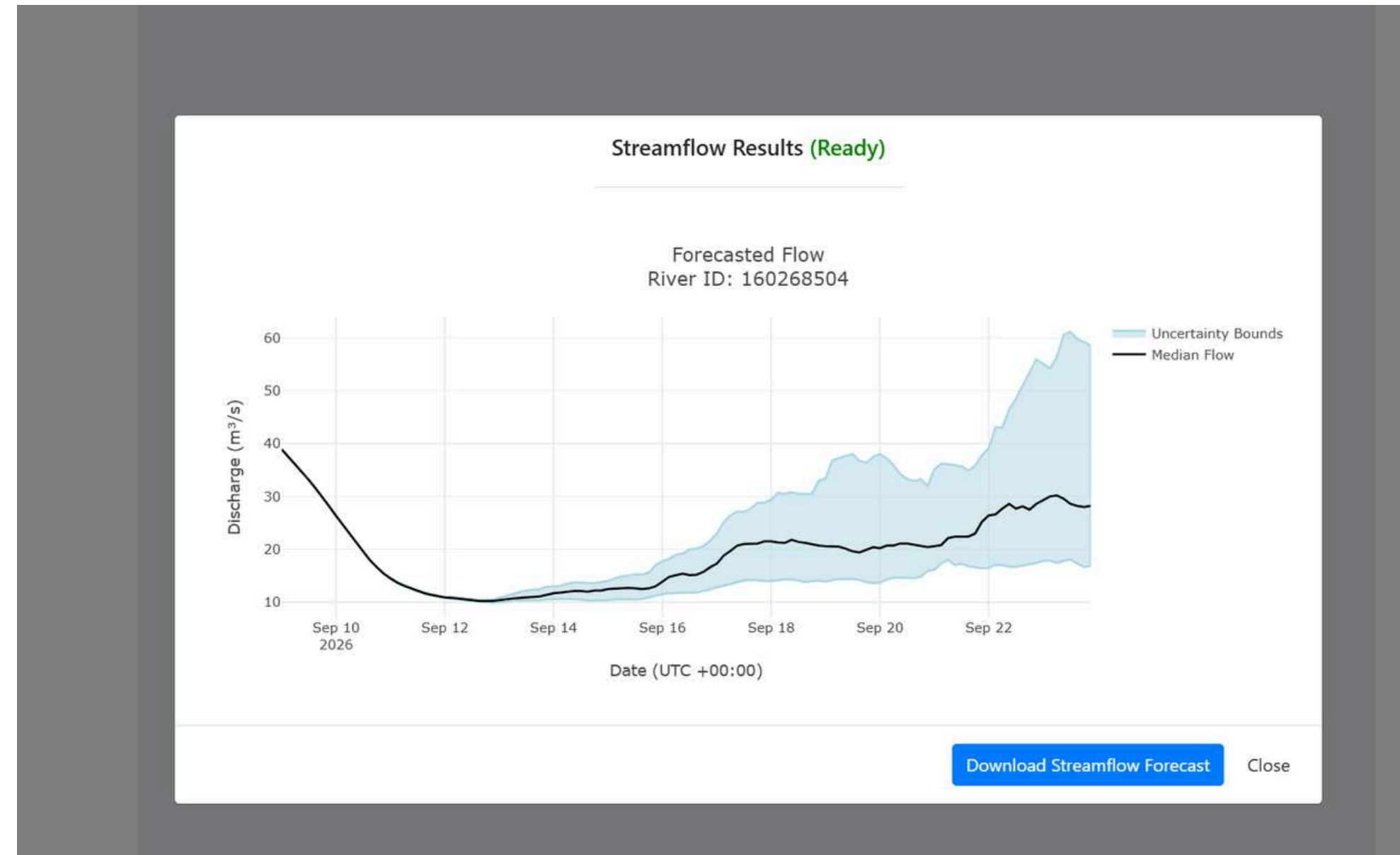
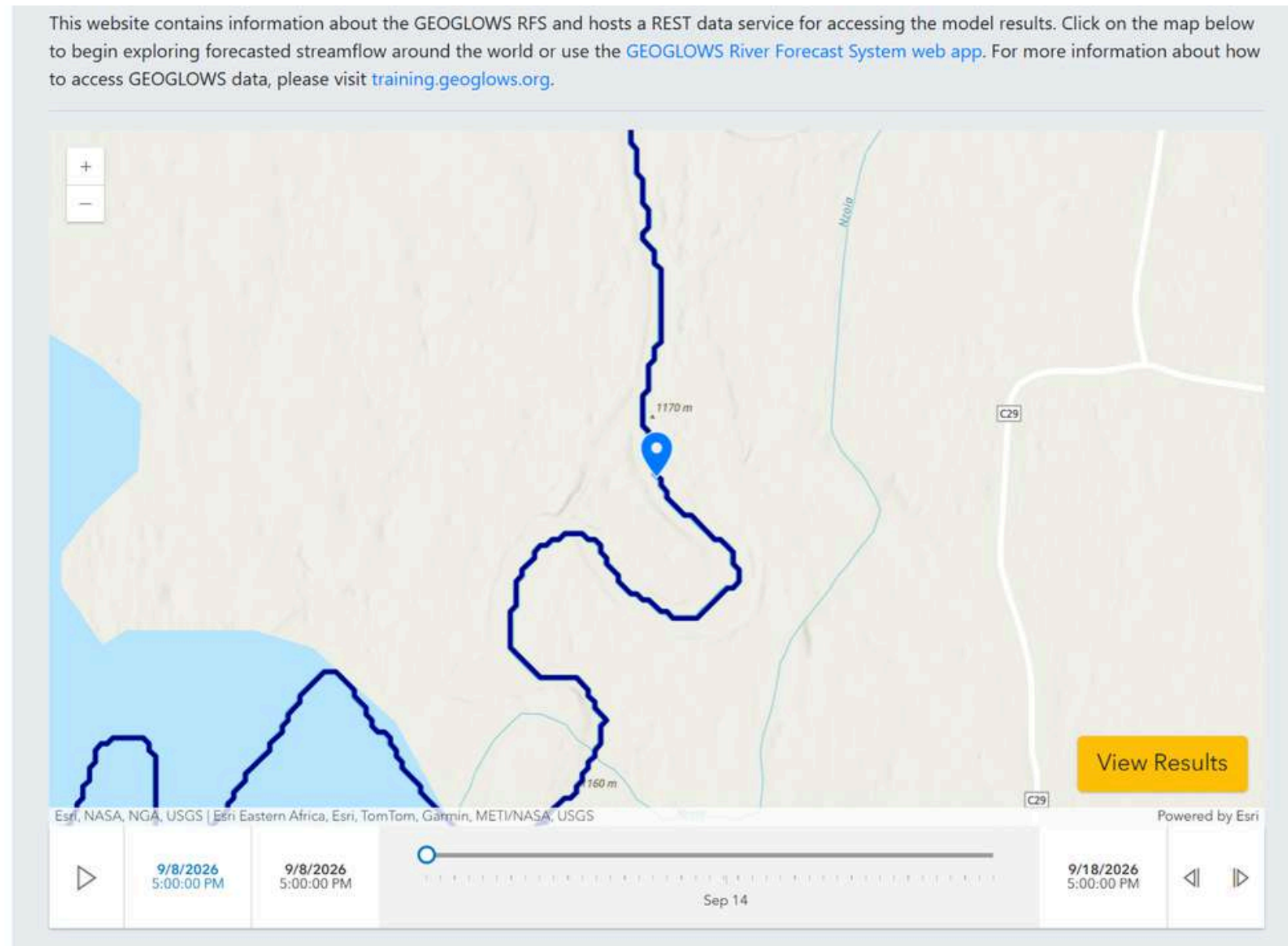
- Nzoia River near its outlet to Lake Victoria
- Approx. coordinates: 0.569° N,
 34.806° E
- GEOGLOWS V2 River ID: 160268504



See the next slide to
learn how to get this ID.



We obtain the GEOGLOWS River ID from the [GEOGLOWS Streamflow Forecast map](#). You can select any location by navigating the map and clicking on a river reach to get its River ID.



You can complete the same exercise for any river you are interested in, such as the **Nyando** or **Sondu Rivers**. Simply select the river in the GEOGLOWS map and use its corresponding River ID.

Access Exercise 4

The fourth exercise can be accessed through the following link for each country:



Kenya: [Access Exercise 4](#)



Rwanda: [Access Exercise 4](#)



Uganda: [Access Exercise 4](#)



Ethiopia: [Access Exercise 4](#)

You may be asked to sign in to your Google account.



Getting Started with HydroServer Programmatically

Python code

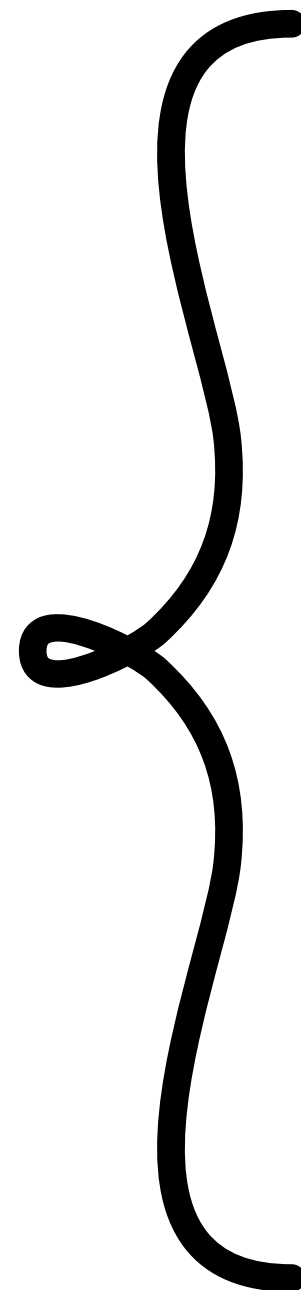
REMINDER

Install
hydroserverpy
package



```
!pip install hydroserverpy==1.11.3
```

Import the **Python packages** needed for the exercise



```
# Import HydroServer to connect to and manage HydroServer resources  
from hydroserverpy import HydroServer
```

```
# Import pandas to read, organize, and process the sensor data  
import pandas as pd
```

```
# Import datetime to work with dates and times  
from datetime import datetime
```

```
# Import getpass to securely enter your HydroServer password  
from getpass import getpass
```

These steps are the same as in the previous exercises.

Getting Started with HydroServer Programmatically

```
Collecting dnspython>=2.0.0 (from email-validator>=2.0.0->pydantic[email]>=2.0->hydroserverpy==1.11.3)
  Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Downloading hydroserverpy-1.11.3-py3-none-any.whl (113 kB)
  113.4/113.4 kB 5.6 MB/s eta 0:00:00
Downloading croniter-6.2.4-py3-none-any.whl (46 kB)
  46.7/46.7 kB 3.3 MB/s eta 0:00:00
Downloading jmespath-1.1.0-py3-none-any.whl (20 kB)
Downloading python_crontab-3.4.0-py3-none-any.whl (27 kB)
Downloading email_validator-2.3.0-py3-none-any.whl (35 kB)
Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
  331.1/331.1 kB 18.2 MB/s eta 0:00:00
Installing collected packages: python-crontab, jmespath, dnspython, email-validator, croniter, hydroserverpy
Successfully installed croniter-6.2.4 dnspython-2.8.0 email-validator-2.3.0 hydroserverpy-1.11.3 jmespath-1.1.0 python-crontab-3.4.0
```



When the packages have been successfully installed and imported, you will see these messages!

[4]
✓ 0s

```
# Import HydroServer to connect to and manage HydroServer resources
from hydroserverpy import HydroServer

# Import pandas to read, organize, and process the sensor data
import pandas as pd

# Import datetime to work with dates and times
from datetime import datetime

# Import getpass to securely enter your HydroServer password
from getpass import getpass
```



Getting Started - Import the ETL Components

Import the
hydroserverpy
components
needed to
build and run
the ETL
pipeline.



```
from hydroserverpy.etl import ETLPipeline
from hydroserverpy.etl.extractors import HTTPExtractor
from hydroserverpy.etl.transformers import (
    CSVTransformer,
    JSONTransformer,
    ETLDataMapping,
    ETLTargetPath
)
from hydroserverpy.etl.operations import ArithmeticExpressionOperation
from hydroserverpy.etl.loaders import HydroServerLoader
```



[4]
✓ 0s

```
from hydroserverpy.etl import ETLPipeline
from hydroserverpy.etl.extractors import HTTPExtractor
from hydroserverpy.etl.transformers import (
    CSVTransformer,
    JSONTransformer,
    ETLDataMapping,
    ETLTargetPath
)
from hydroserverpy.etl.operations import ArithmeticExpressionOperation
from hydroserverpy.etl.loaders import HydroServerLoader
```



Programmatically – Set the Initial Parameters for Login

Set the initial parameters to connect:

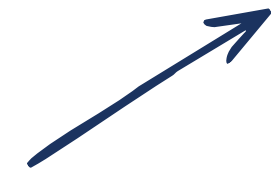
- Host
- E-mail
- Password



```
# Set initial parameters to connect to HydroServer
```

```
hydroserver_host = 'https://playground.hydroserver.org'
```

HydroServer Playground URL



```
# Change the email and password below to your HydroServer username and password
```

```
hydroserver_email = 'svicario@lincolninst.edu'
```

```
hydroserver_password = getpass('Enter your HydroServer password: ')
```

IMPORTANT!

When you run the code, you will be prompted to enter your password.



```
# Set initial parameters to connect to HydroServer
hydroserver_host = 'https://playground.hydroserver.org'

# Change the email and password below to your HydroServer username and password
hydroserver_email = 'svicario@lincolninst.edu' #'user@youremail.com'
hydroserver_password = getpass('Enter your HydroServer password: ') #getpass('Enter your HydroServer password: ')
```

Enter your HydroServer password:



These steps are the same as in the previous exercises.

Reminder

Programmatically – Log in into HydroServer

Reminder

The next step is to **establish a connection with HydroServer** using the parameters you just set up.

→

```
# Initialize HydroServer connection with credentials.  
hs = HydroServer(  
    host='https://playground.hydroserver.org',  
    email='your@email.com',  
    password='yourpassword'  
)  
print('\nSuccessfully connected to HydroServer!')
```

```
# Initialize HydroServer connection with credentials.  
hs = HydroServer(  
    host=hydroserver_host,  
    email=hydroserver_email,  
    password=hydroserver_password  
)  
  
print('\nSuccessfully connected to HydroServer!')
```

...

Successfully connected to HydroServer!



If the connection is successful, you will see this message in your console

These steps are the same as in the previous exercises.

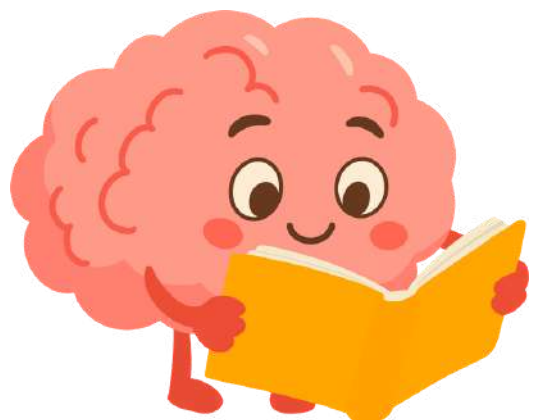
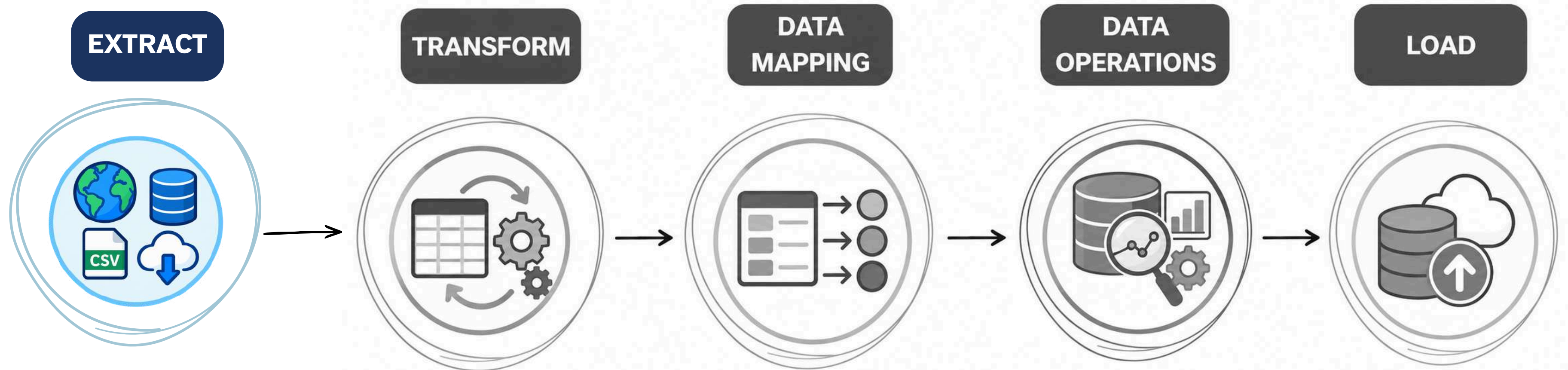
As in the previous exercises, before loading the GEOGLOWS observations, we need to **create** the **monitoring site** and **datastream** where the **Nzoia River forecast data will be stored.**

We will run the Python code together during the training, so there is no need to go through each step in detail here.

For detailed instructions on a similar process, please refer to the **Session 5a presentation.**

HydroServer Extract-Transform-Load (ETL) Pipeline

Now that the datastream is ready, we start with the EXTRACT step, where we retrieve data from the external Web API using Python.



Extract: Where am I getting the data from?

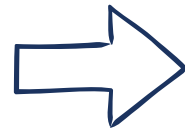


hydroserverpy provides different extractors for retrieving data from different sources, such as HTTP, FTP, or local files.

Extractor	Gets data from	Example
 HTTPExtractor	Web/API	GEOGLOWS API
FTPExtractor	FTP server	File hosted on an FTP server
LocalFileExtractor	Local filesystem	CSV on your computer/server

Extract: Choosing a hydrosurveypy Extractor

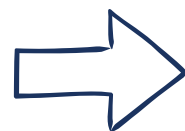
Web/API



```
from hydrosurveypy.etl.extractors import HTTPExtractor

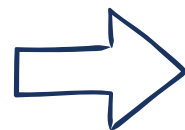
extractor = HTTPExtractor(
    source_uri="https://api.example.com/data/{station_id}/export.csv"
)
```

FTP



```
extractor = FTPExtractor(
    host="ftp.example.com",
    filepath="/data/{station_id}/readings.csv",
    username="user",
    password="pass",
    port=21,
)
```

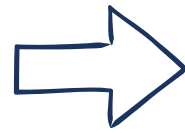
Local File



```
extractor = LocalFileExtractor(
    source_uri="/mnt/data/{station_id}/readings.csv"
)
```

Extract: Choosing the Right Extractor

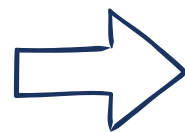
HTTP



```
from hydrosERVERpy.etl.extractors import HTTPExtractor

extractor = HTTPExtractor(
    source_uri="https://api.example.com/data/{station_id}/export.csv"
)
```

FTP



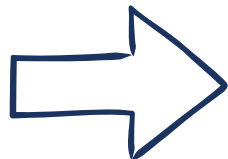
```
extractor = FTPExtractor(
    host="ftp.example.com",
    filepath="/data/{station_id}/readings.csv",
    username="user",
    password="pass",
    port=21,
)
```

Local File



```
extractor = LocalFileExtractor(
    source_uri="/mnt/data/{station_id}/readings.csv"
)
```


Step 1: Extract – Connect to the GEOGLOWS API

For GEOGLOWS,
we retrieve data
from a Web API, so 
we use the
HTTPExtractor.

```
# GEOGLOWS V2 river reach nearest gauge 1DA02 on the Nzoia River  
GEOGLOWS_RIVER_ID = 160268504
```

```
forecast_url = (  
    f"https://geogloWS.ecmwf.int/api/v2/"  
    f"forecast/{GEOGLOWS_RIVER_ID}?format=csv"  
)
```

GEOGLOWS API



```
extractor = HTTPExtractor(source_uri=forecast_url)
```

```
print("Extractor created.")  
print(f"Source: {forecast_url}")
```

```
... Extractor created.  
Source: https://geogloWS.ecmwf.int/api/v2/forecast/160193736?format=csv
```

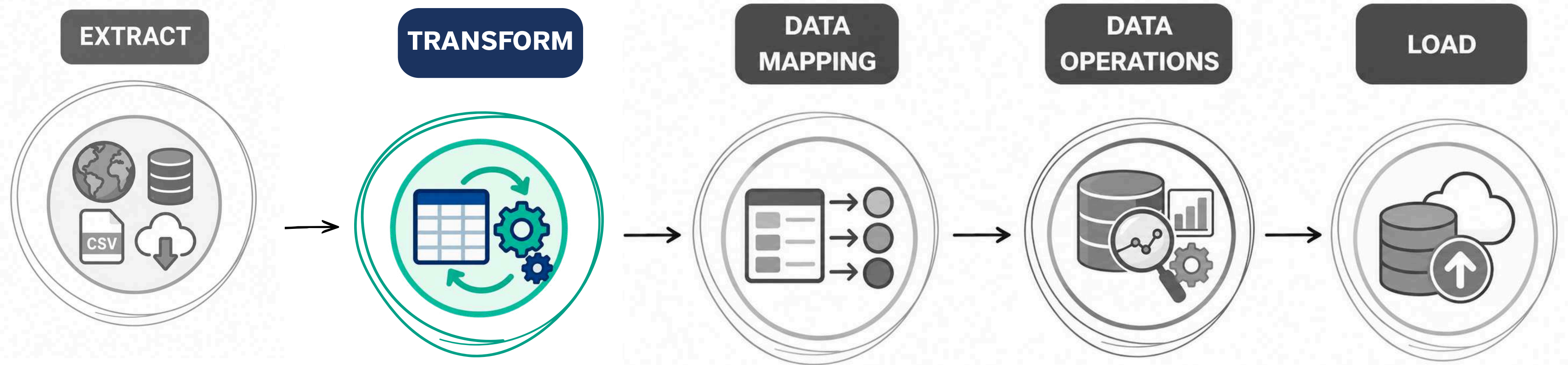


**“Extractor created” means that you have configured the part of the ETL
pipeline that knows where and how to retrieve the data.**



HydroServer Extract-Transform-Load (ETL) Pipeline

Once the data are retrieved, we move to the TRANSFORM step, where the extracted data are converted into a DataFrame that can be processed and prepared for HydroServer.



Transform: How should I read the data?

Transformers parse the raw data retrieved by the Extractor and convert it into a DataFrame (a table of rows and columns).

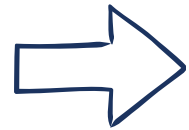
The transformation depends on the format of the source data:

- CSV: The data are already organized in rows and columns, so they can be converted directly into a DataFrame.
- JSON: The data may be nested or hierarchical, so the Transformer extracts the relevant fields and organizes them into rows and columns.

Transform: Choosing the Right Transformer

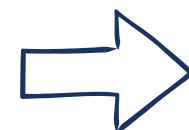
```
from hydrosERVERpy.etl.transformers import CSVTransformer
```

CSV



```
transformer = CSVTransformer(  
    timestamp_key="datetime",  
    delimiter=";",  
    header_row=1,  
    data_start_row=2,  
)
```

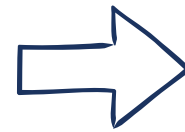
JSON



```
transformer = JSONTransformer(  
    timestamp_key="timestamp",  
    jmespath="response.data",  
)
```


Step 2: Transform – Convert the GEOGLOWS CSV Data

The GEOGLOWS API returns the forecast as a CSV file, therefore we choose the CSV transformer



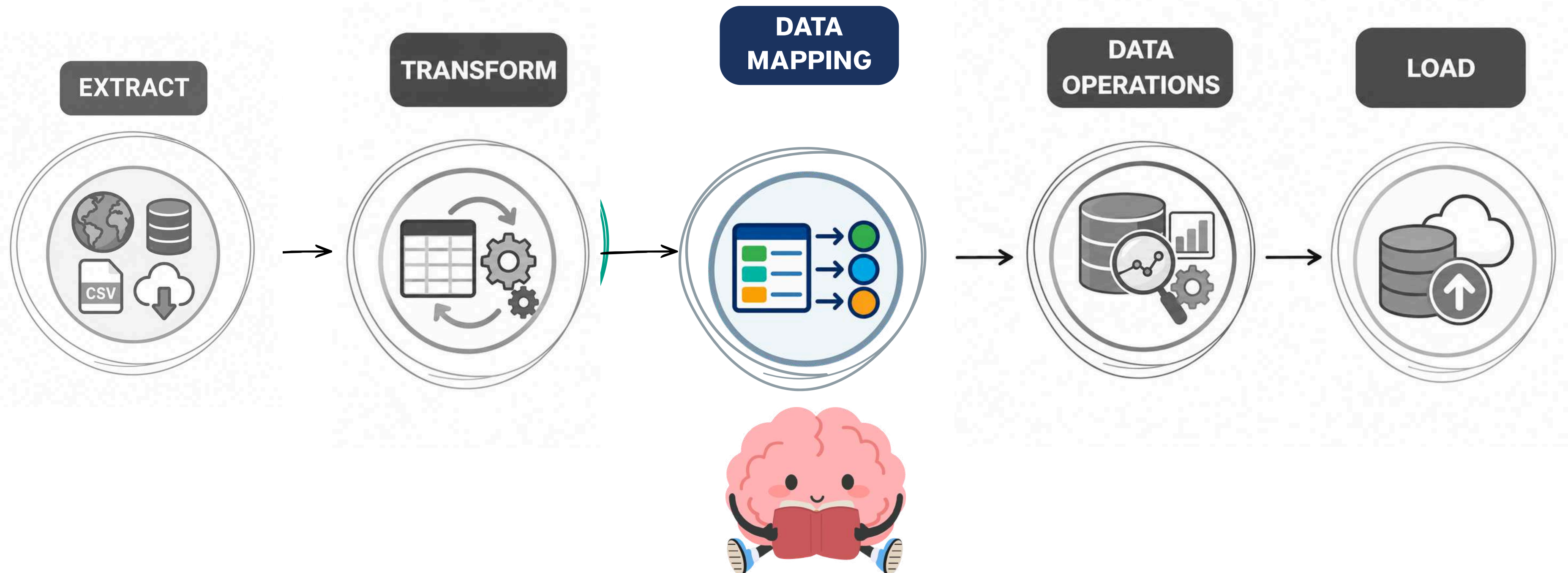
```
transformer = CSVTransformer(  
    timestamp_key="datetime",  
    delimiter=";",  
    header_row=1,  
    data_start_row=2  
)  
  
print("Transformer created.")
```

```
... Transformer created.
```



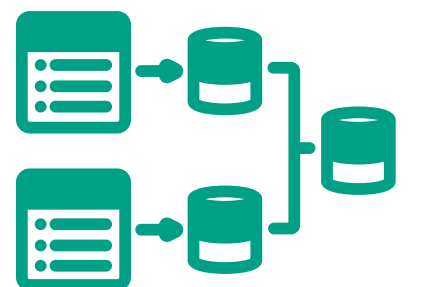
HydroServer Extract-Transform-Load (ETL) Pipeline

Next, we move to the DATA MAPPING step, where the columns in the DataFrame are mapped to the corresponding HydroServer observation fields.



Data Mapping: Where should the data go?

- Data mappings define where each variable in your data will be stored in HydroServer.
- They connect each source variable to the corresponding HydroServer datastream.



Step 3: Data Mapping – Connect the Forecast to the Datastream

Variable/column coming from the GEOGLOWS data. →

Identifier of the target Datastream we just created →

```
data_mappings = [  
    ETLDataMapping(  
        source_identifier="flow_median",  
        target_paths=[  
            ETLTargetPath(  
                target_identifier=str(forecast_datastream.uid)  
            )  
        ]  
    )  
]
```

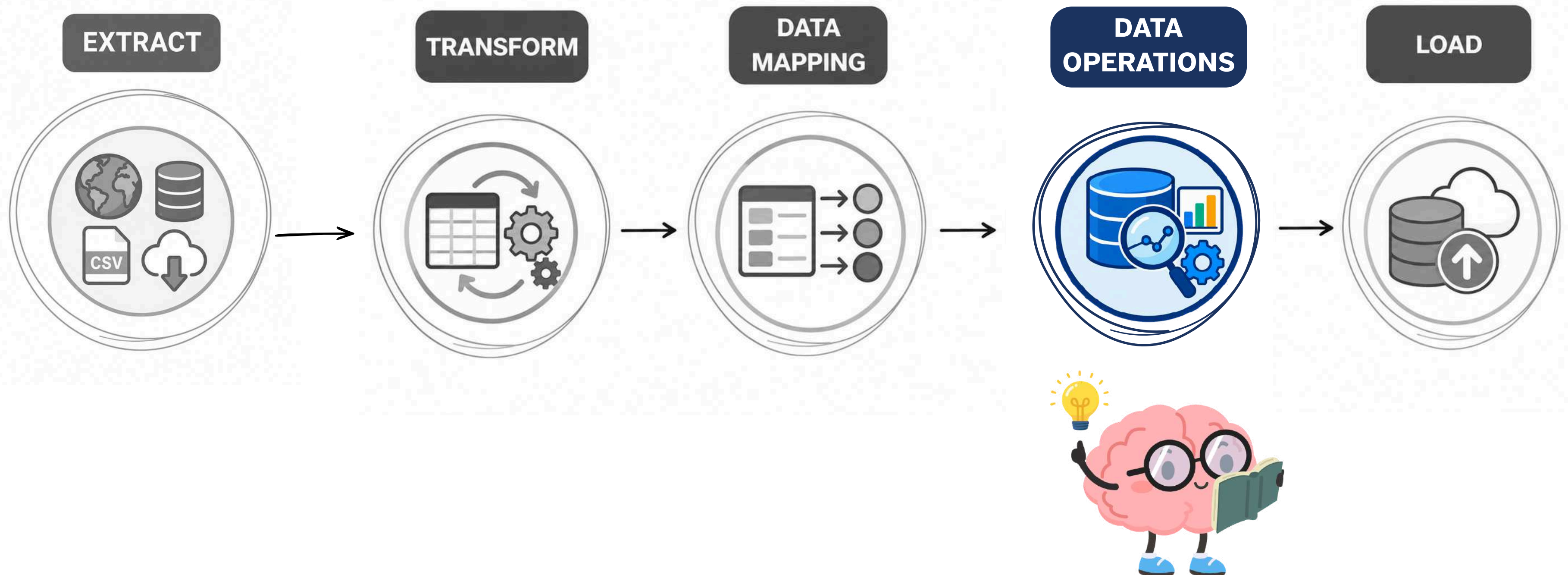
```
print("Data mapping created.")
```

```
... Data mapping created.
```



HydroServer Extract-Transform-Load (ETL) Pipeline

Once the data are mapped, we can perform Data Operations to modify, calculate, or process the data before loading them into HydroServer.



Data Operations

- We can modify data before loading it into HydroServer.
- Multiple operations can be applied one after another.
- Operations include:
 - Arithmetic Calculations
 - Time aggregation

We are not going to perform any Data Operations in Exercise 4.



Example: Data Operations

IMPORTANT!

- This is just an example. We are not going to perform any Data Operations in Exercise 4.

```
from hydrosolverpy.etl.transformers import ETLTargetPath
```

UUID of the
HydroServer
datastream

```
ETLTargetPath(  
    target_identifier="<datastream-uuid>",  
    data_operations=[  
        ArithmeticExpressionOperation(  
            expression="(x - 32) / 1.8",    # Fahrenheit to  
            Celsius  
            target_identifier="<datastream-uuid>",  
        ),  
    ],  
)
```

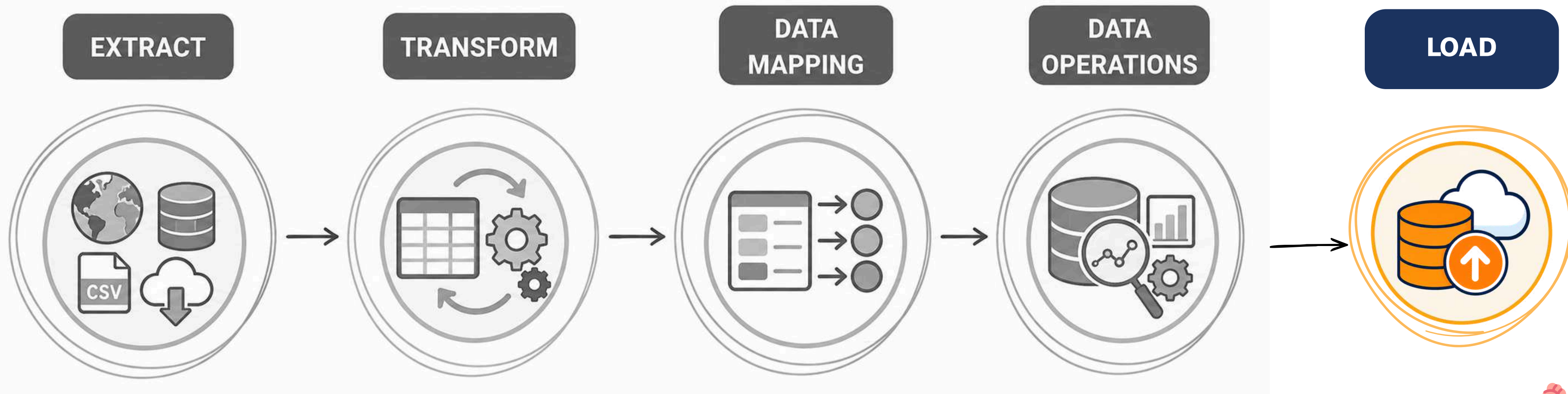
Before loading the
data, apply the
following
operation(s).



**Just an example of how to
perform operations on your
observations before loading
them into HydroServer.s!**

HydroServer Extract-Transform-Load (ETL) Pipeline

Finally, the Loader uploads the observations to the corresponding HydroServer datastreams defined in the Data Mapping step.



Step 4: Load – Create the HydroServer Loader

Configure the
Loader to upload
data in batches of
5,000

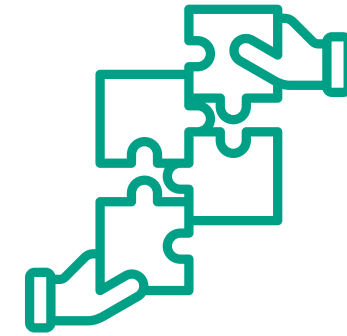


```
loader = HydroServerLoader(  
    client=hs,  
    chunk_size=5000  
)  
  
print("Loader created.")
```

```
... Loader created.
```



Assemble the ETL Pipeline



- Combines the ETL components we just created-
Extractor, Transformer, Data Mapping, and Loader-
into a single workflow.
- Prepares the ETL pipeline for execution. No data are
retrieved or uploaded until `pipeline.run()` is called.

Step 5: Assemble the ETL Pipeline

Combine the Extractor,
Transformer, and
Loader into a single
ETL pipeline



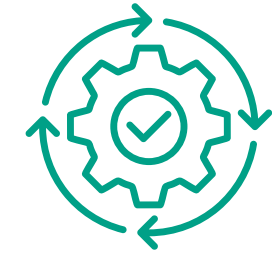
```
# Assemble the ETL pipeline
pipeline = ETLPipeline(
    extractor=extractor,
    transformer=transformer,
    loader=loader
)

print("ETL pipeline created.")
```

```
... ETL pipeline created.
```



Run the Pipeline: Ready to execute



- Runs the complete ETL pipeline using the data mappings, retrieving the GEOGLOWS forecast, transforming it, and uploading it to HydroServer.

Step 6: Run the pipeline

Run the complete ETL
pipeline using the data
mappings →

```
# Run the pipeline: retrieve the latest GEOGLWS forecast and upload it to HydroServer
context = pipeline.run(
    data_mappings=data_mappings,
    raise_on_error=False
)
```

Check the pipeline status
and confirm that the
observations were
successfully loaded →

```
print(f"Pipeline status: {context.status}")
print(f"Final stage: {context.stage}")
```

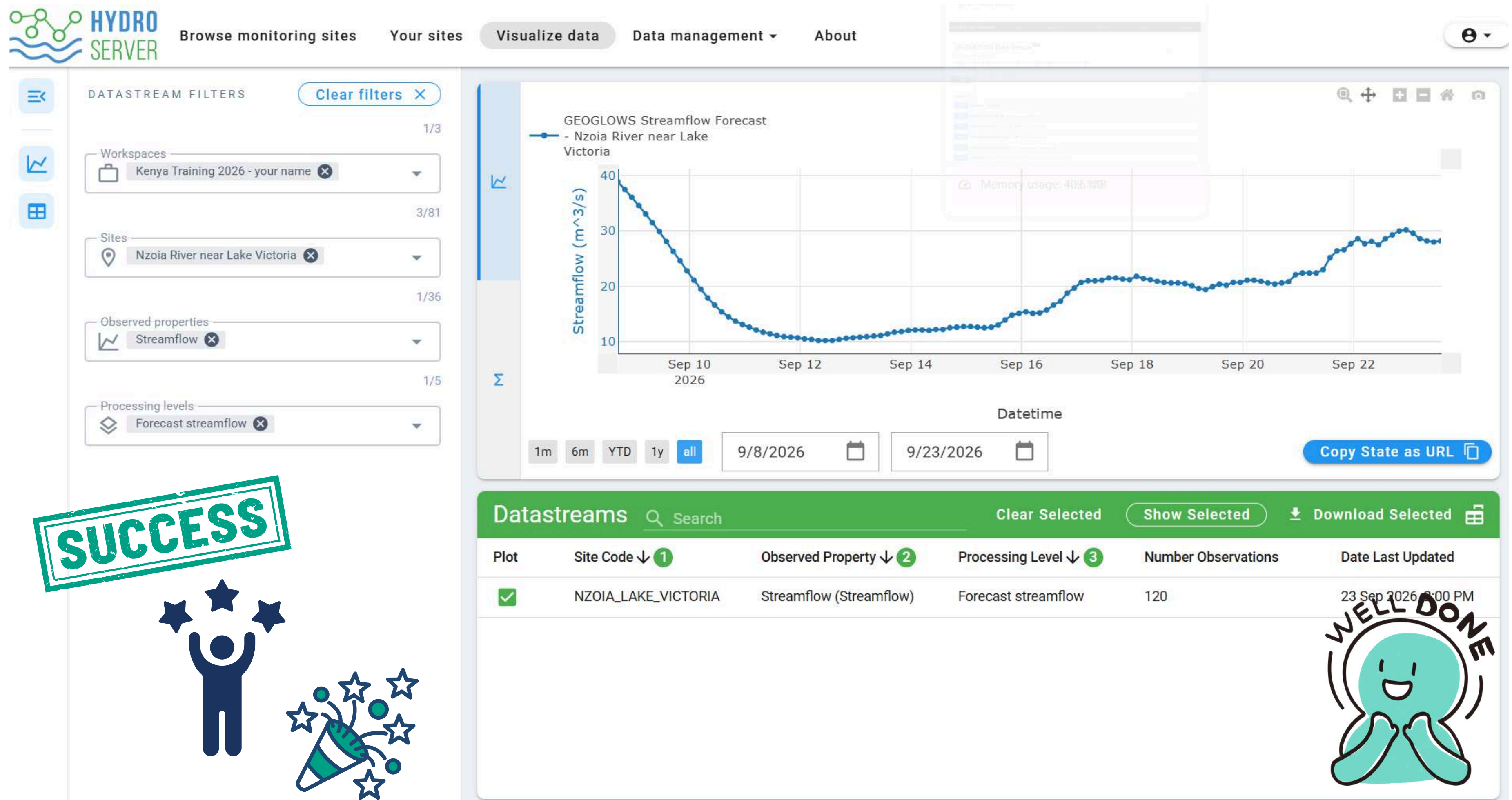
```
if context.results is not None:
    print(context.results)
```

```
# If the run is successful, the latest GEOGLWS forecast values are now
# stored in the HydroServer forecast datastream created above.
```

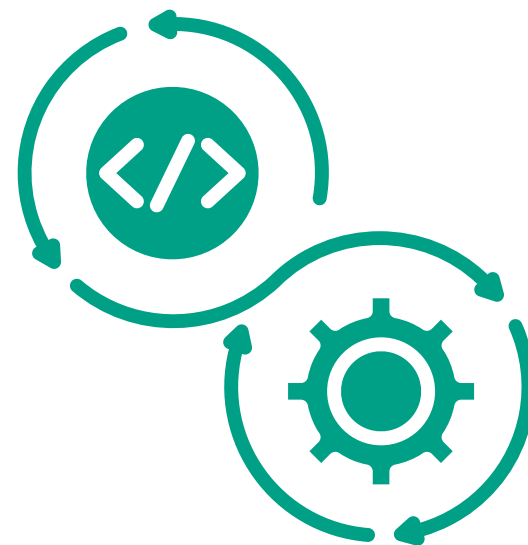
```
INFO:hydroserverpy.etl.models.pipeline:Starting extract
INFO:hydroserverpy.etl.models.pipeline:Resolved runtime source URI: https://geoglwes.ecmwf.int/api/v2/forecast/160193736?format=csv
INFO:hydroserverpy.etl.extractors.http:Requesting data from source URI
INFO:hydroserverpy.etl.models.pipeline:Extractor returned payload: {'type': 'BytesIO', 'bytes': 4627}
INFO:hydroserverpy.etl.models.pipeline:Starting transform
INFO:hydroserverpy.etl.models.pipeline:Transform result: {'type': 'DataFrame', 'rows': 120, 'columns': 2, 'datastreams': 1}
INFO:hydroserverpy.etl.models.pipeline:Starting load
INFO:hydroserverpy.etl.loaders.hydroserver:Load result: loaded=120 available=120 cutoff=None
Pipeline status: SUCCESS
Final stage: CLEANUP
success_count=1 failure_count=0 skipped_count=0 values_loaded_total=120 earliest_timestamp=None latest_timestamp=None target_results={'01a06812-7445-7514-aa7f-3336b07481d8':
```



If you go to the HydroServer Dashboard, you can now see the GEOGLOWS streamflow forecast!



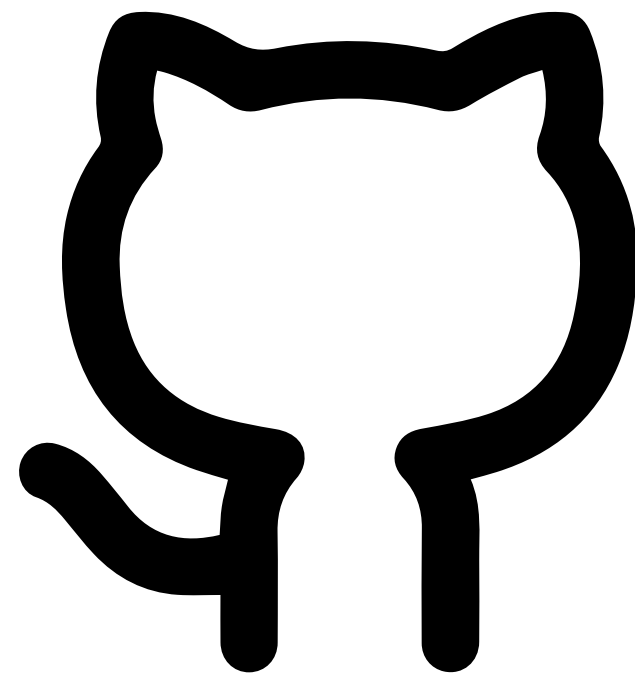
Once you have successfully run the pipeline, you can schedule it to run automatically (for example, every day) using a **job orchestration system**.



For instance, on Windows, you can use Windows Task Scheduler to automate the pipeline. See Slides 78–91 for step-by-step instructions. The process is quite straightforward!

Also, GitHub Actions can be used to **schedule** the ETL script to run automatically, regularly retrieve new GEOGLOWS forecasts, and load them into HydroServer.

Learn more by clicking this [link](#).

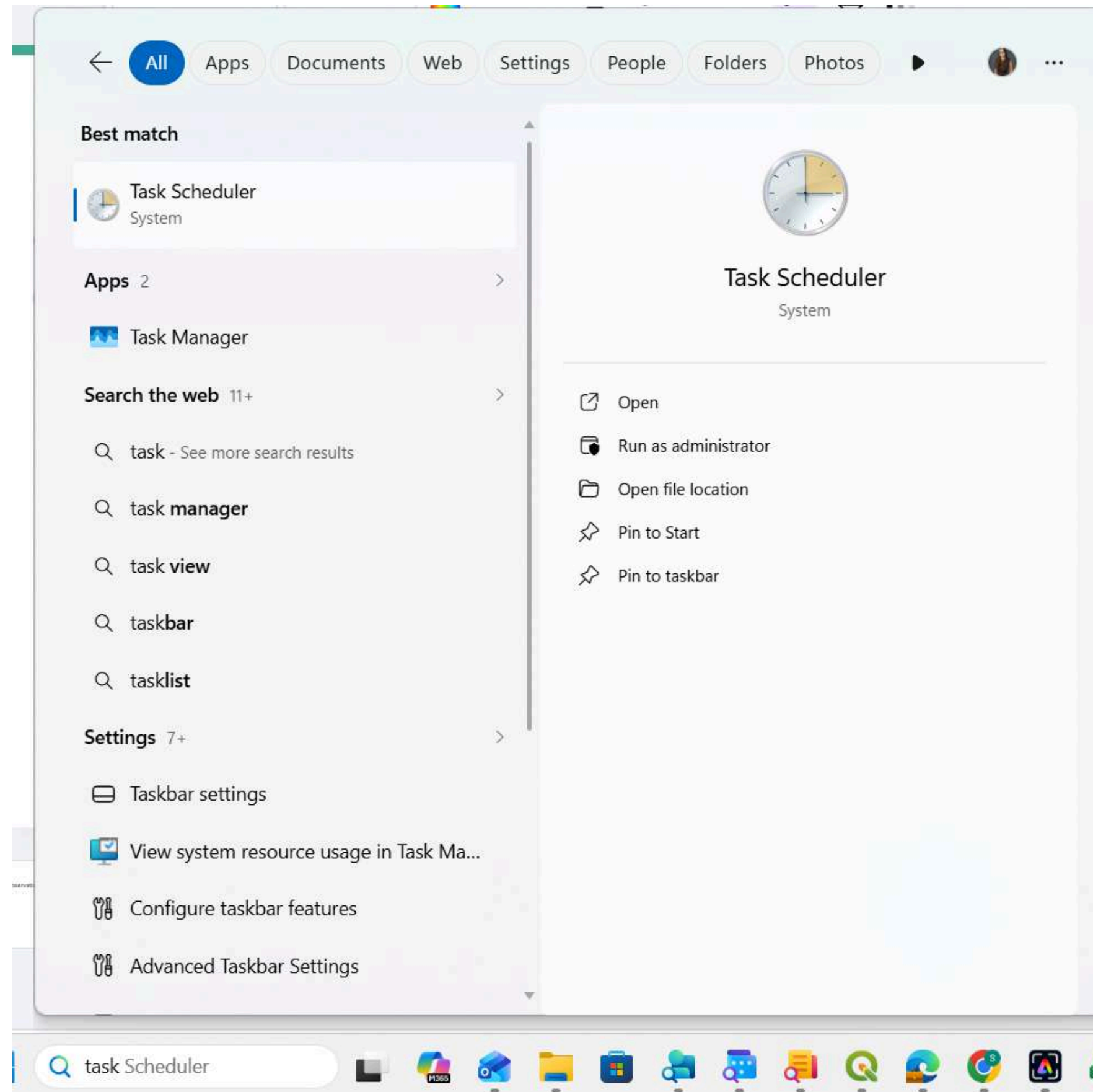




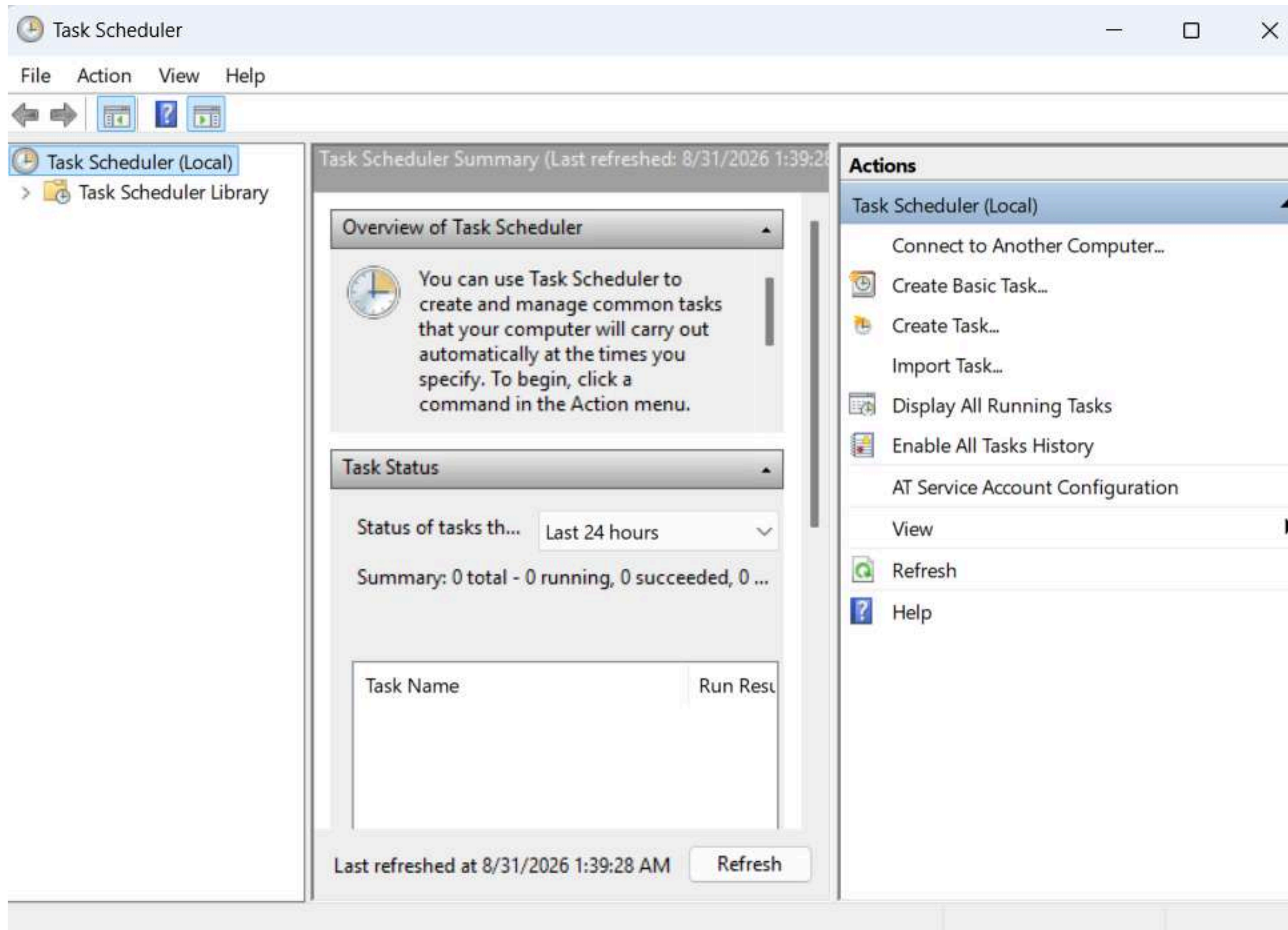
Job Orchestration in Windows



Search for “Task Scheduler” in Windows.



Click on "Create Basic Task..."



Give the task a name.

Create Basic Task Wizard

 Create a Basic Task

Create a Basic Task

Trigger

Action

Finish

Use this wizard to quickly schedule a common task. For more advanced options or settings such as multiple task actions or triggers, use the Create Task command in the Actions pane.

Name: GEOGLOWS HydroServer ETL

Description:

< Back

Next >

Cancel

Set how often you want the task to run.

Create Basic Task Wizard ✕

 Task Trigger

Create a Basic Task

Trigger

Action

Finish

When do you want the task to start?

☒ Daily

☐ Weekly

☐ Monthly

☐ One time

☐ When the computer starts

☐ When I log on

☐ When a specific event is logged

< Back

Next >

Cancel

Indicate when you want the task to start.

Create Basic Task Wizard

 Daily

Create a Basic Task

Trigger

Daily

Action

Finish

Start: 9/ 9/2026 5:54:30 AM ☐ Synchronize across time zones

Recur every: 1 days

< Back Next > Cancel

Select "Start a program."

Create Basic Task Wizard

 Action

Create a Basic Task

Trigger
Daily

Action

Finish

What action do you want the task to perform?

☒ Start a program

☐ Send an e-mail (deprecated)

☐ Display a message (deprecated)

< Back Next > Cancel

Browse to the location where you saved the program.

Create Basic Task Wizard ✕

 Start a Program

Create a Basic Task

Trigger

Daily

Action

Start a Program

Finish

Program/script:

C:\Users\SaraVicario\Downloads\Nzoia_GEOGLOWS_HydroServer_Foreca Browse...

Add arguments (optional):

Start in (optional):

< Back Next > Cancel

Then, click "Finish."

Create Basic Task Wizard ✕

 Summary

Create a Basic Task

Trigger

Daily

Action

Start a Program

Finish

Name: GEOGLOWS HydroServer ETL

Description:

Trigger: Daily; At 1:45 AM every day

Action: Start a program; C:\Users\SaraVicario\Downloads\Nzoia_GEOGLOWS_HydroSer

☐ Open the Properties dialog for this task when I click Finish

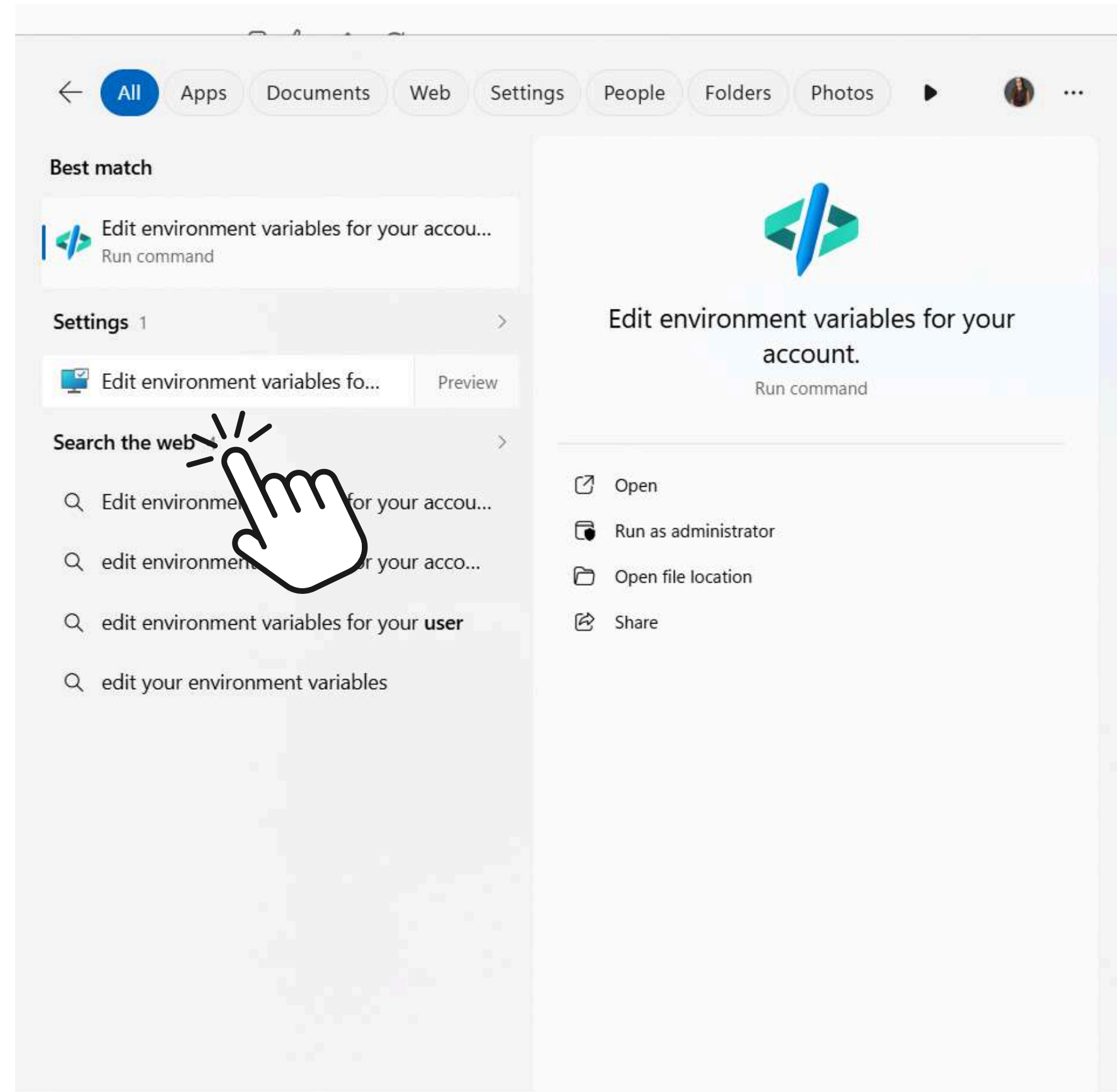
When you click Finish, the new task will be created and added to your Windows schedule.

< Back Finish Cancel

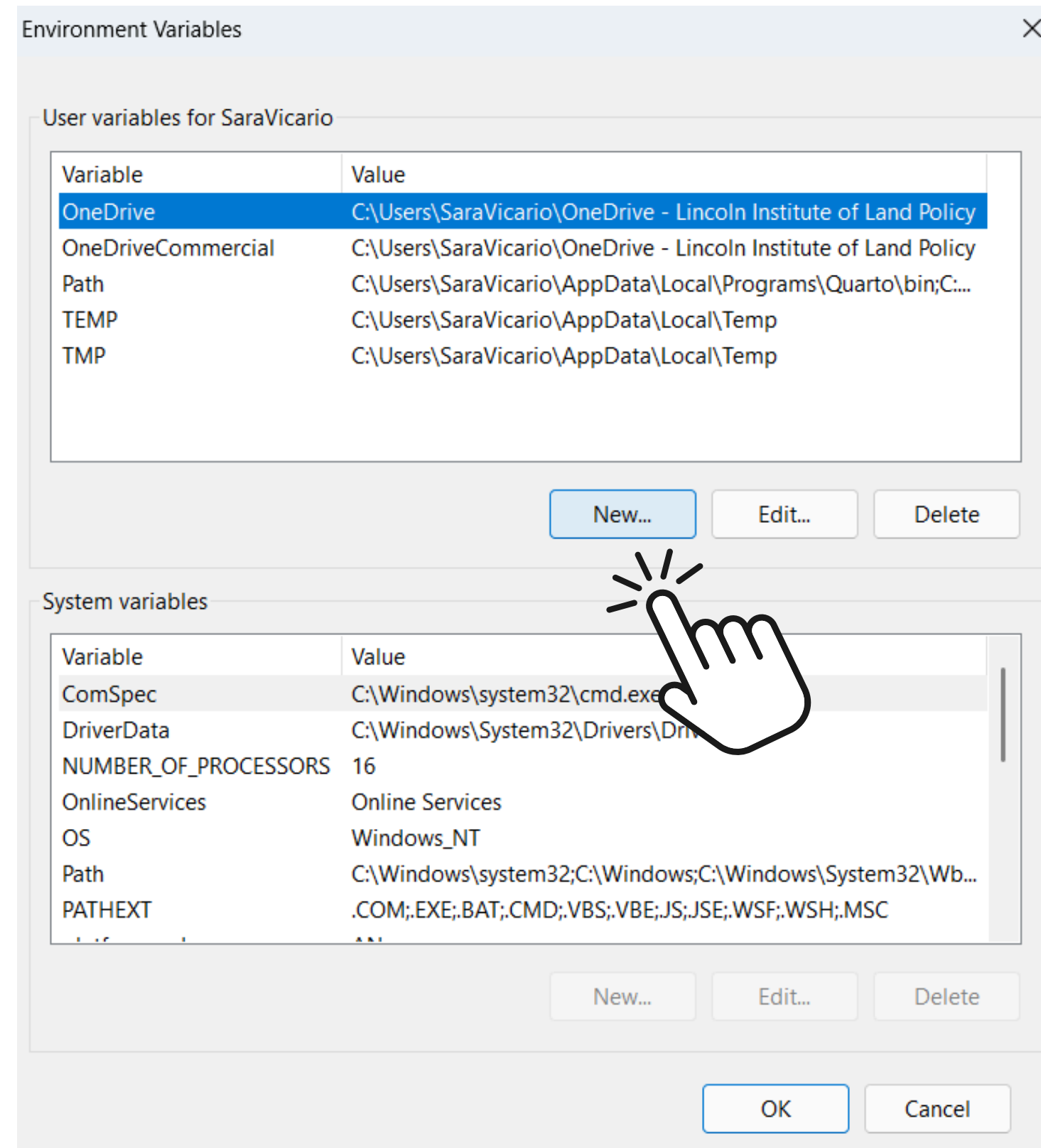
Next, you need to set up your HydroServer credentials as environment variables.

This allows the program to access HydroServer automatically without entering your email and password each time it runs.

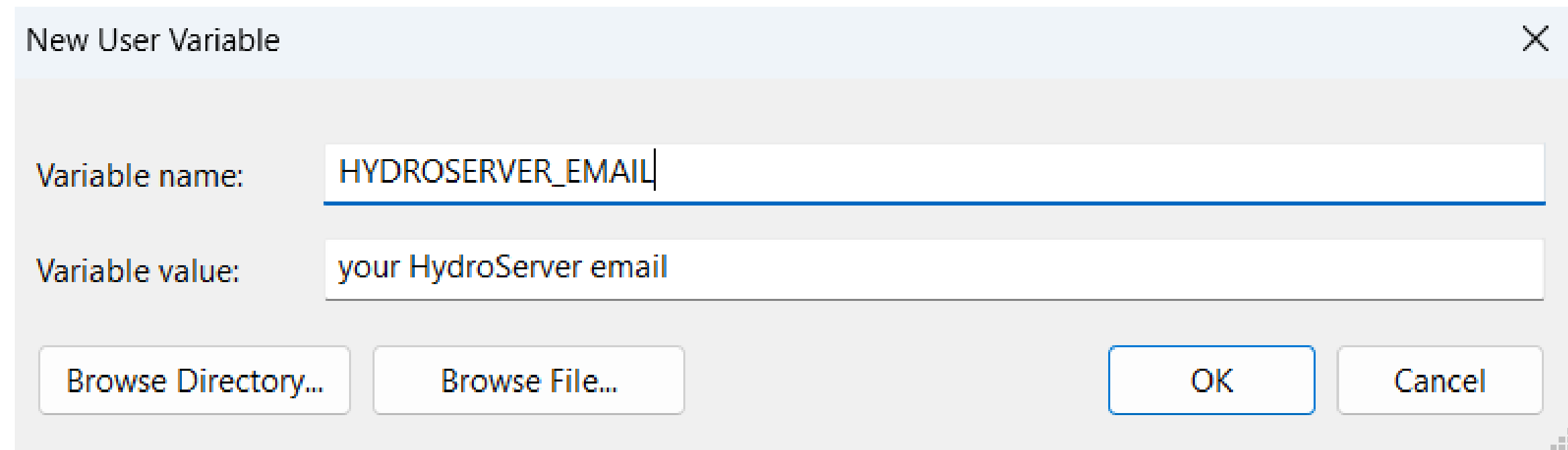
Search for “Edit environment variables for your account” in Windows.



Click "New..."



Create two new user variables: one for your HydroServer email and one for your password.



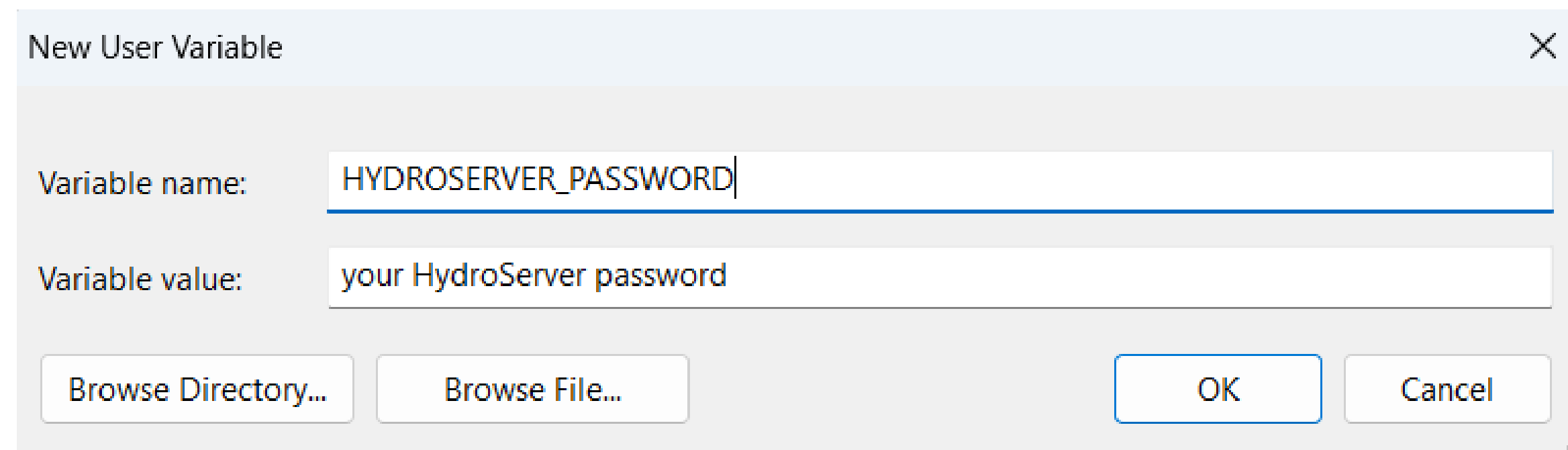
A screenshot of a 'New User Variable' dialog box. The title bar is light blue with a close button (X) on the right. The main area has a light gray background. There are two text input fields: the first is labeled 'Variable name:' and contains the text 'HYDROSERVER_EMAIL'; the second is labeled 'Variable value:' and contains the text 'your HydroServer email'. Below the input fields are four buttons: 'Browse Directory...', 'Browse File...', 'OK', and 'Cancel'. The 'OK' button is highlighted with a blue border.

New User Variable

Variable name: HYDROSERVER_EMAIL

Variable value: your HydroServer email

Browse Directory... Browse File... OK Cancel



A screenshot of a 'New User Variable' dialog box, similar to the one above. The title bar is light blue with a close button (X) on the right. The main area has a light gray background. There are two text input fields: the first is labeled 'Variable name:' and contains the text 'HYDROSERVER_PASSWORD'; the second is labeled 'Variable value:' and contains the text 'your HydroServer password'. Below the input fields are four buttons: 'Browse Directory...', 'Browse File...', 'OK', and 'Cancel'. The 'OK' button is highlighted with a blue border.

New User Variable

Variable name: HYDROSERVER_PASSWORD

Variable value: your HydroServer password

Browse Directory... Browse File... OK Cancel

You need to update the connection code to use the environment variables you just created instead of entering your email and password directly.

You can find this code commented out in the exercise.

Connect to HydroServer Using Environment Variables (OPTION 2)

For an automated ETL workflow, the HydroServer credentials should not require manual input each time the script runs.

Store your HydroServer email and password as Windows environment variables named `HYDROSERVER_EMAIL` and `HYDROSERVER_PASSWORD`. The Python script can then retrieve these credentials automatically when it is executed by Windows Task Scheduler.

```
: import os

# Retrieve HydroServer credentials from Windows environment variables
hydroserver_email = os.environ["HYDROSERVER_EMAIL"]
hydroserver_password = os.environ["HYDROSERVER_PASSWORD"]

# Connect to HydroServer
hydroserver_host = "https://playground.hydroserver.org"

hs = HydroServer(
    host=hydroserver_host,
    email=hydroserver_email,
    password=hydroserver_password
)

print("Successfully connected to HydroServer using environment variables.")
```



Code adapted to read your email and password as Windows environment variables.

This code is included as a comment in Exercise 3.

Now, we are going to automate data ingestion into HydroServer using HydroServer-managed ETL tasks by completing **Exercise 5**.



Exercise 5:

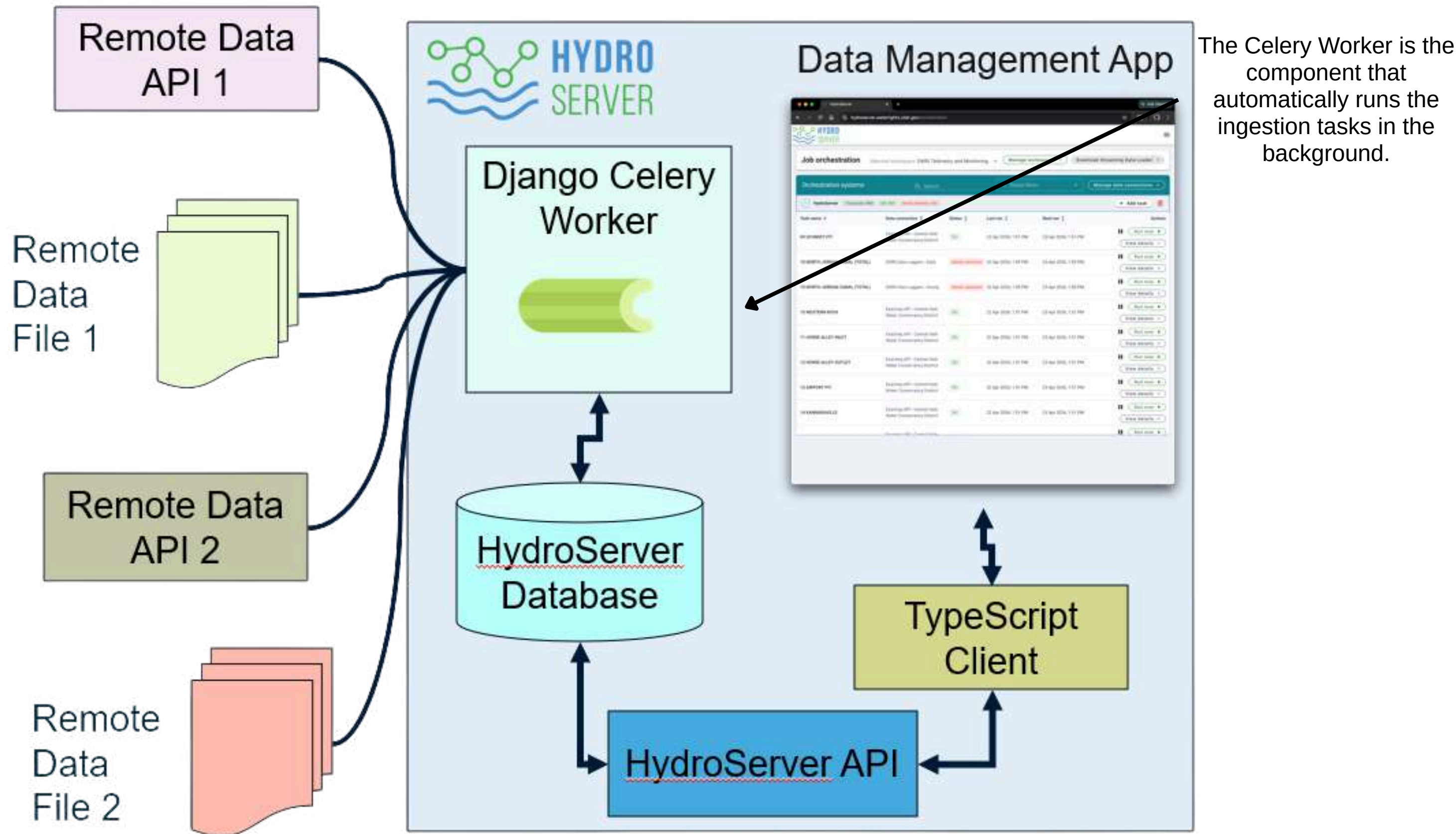
**Automating Data Ingestion into
HydroServer with HydroServer-
Managed Extract-Transform-Load (ETL).
Tasks**

HydroServer also includes a built-in automation system that can connect directly to remote APIs or data files, automatically retrieve new observations, and load them into HydroServer.



This provides another option for automating data ingestion without having to run your own external Python workflow.

HydroServer can manage automated data ingestion on the server. A background worker periodically retrieves data from configured remote APIs or files, processes the data, and loads the observations directly into HydroServer.





- **Note:** This functionality is not currently supported on the public HydroServer Playground and requires a deployed HydroServer instance.

- **We can still complete this exercise and create the Job Orchestration task, but the task will not run and upload observations on the Playground.**

- On a deployed HydroServer instance with Job Orchestration enabled, the task can run automatically and upload new observations according to the defined schedule.

Access Exercise 5

The fifth exercise can be accessed through the following link for each country:



Kenya: [Access Exercise 5](#)



Rwanda: [Access Exercise 5](#)



Uganda: [Access Exercise 5](#)



Ethiopia: [Access Exercise 5](#)

Step 1: Create a Reusable Data Connection

The Data Connection defines where the data come from and how HydroServer should read them. By using a `river_id` placeholder, the same connection can be reused for different rivers, with the specific River ID provided when creating the ETL Task.

```
# Create a reusable Data Connection to the GEOGLWS forecast API
geoglows_data_connection = hs.dataconnections.create(
    Name of the data connection → name="GEOGLWS Streamflow Forecasts",
    Your Workspace → workspace=workspace_id,
    URL of the external data source (API request URL) → source_url=(
        "https://geoglows-ecmwf.int/api/v2/"
        "forecast/{river_id}?format=csv"
    ),
    format of the response → payload_type="CSV",
    field of the date and time → timestamp_key="datetime",
    where to find the observations → header_row=1,
    data_start_row=2,
    delimiter=","
    placeholder_variables=[
        {
            "name": "river_id",
            "variable_type": "per_task"
        }
    ],
    Values of the placeholders →
    print(f"{geoglows_data_connection.name}: {geoglows_data_connection.uid}")
```

placeholders

We will provide these values when we create the task

Step 2: Create and Schedule the ETL Task

Create an ETL Task that uses the Data Connection, specifies the River ID, maps the GEOGLOWS forecast values to the target HydroServer Datastream, and defines how often the task should run.

Specify the data connection we just created.

Task variables

Define which data from the source goes into which HydroServer datastream.

Run this task every day

Turn this task on.

```
geoglows_etl_task = hs.etltasks.create(  
    name="Daily Nzoia GEOGLOWS Streamflow Forecast",  
    data_connection=geoglows_data_connection,  
    task_variables={  
        "river_id": "160193736"  
    },  
    mappings=[  
        {  
            "source_identifier": "flow_median",  
            "target_datastream_id": str(forecast_datastream.uid),  
        }  
    ],  
    interval=1,  
    interval_period="days",  
    enabled=True,  
)  
  
print(f"{geoglows_etl_task.name}: {geoglows_etl_task.uid}")  
print(f"Enabled: {geoglows_etl_task.enabled}")  
print(f"Next run: {geoglows_etl_task.next_run_at}")
```

When you run the ETL Task, you will see the following message:

```
... Daily Nzoia GEOGLOWS Streamflow Forecast: 01a08711-3198-756d-90c9-d0d318ca3c6d  
Enabled: True  
Next run: 2026-09-10 16:47:26.112074+00:00
```



If you go to the Web Dashboard and click on Job Orchestration, you will see the ETL Task you just created and scheduled



[Browse monitoring sites](#)[Your sites](#)[Visualize data](#)

Data management

Manage metadata

Job orchestration

[About](#)

Job orchestration

Selected workspace: Kenya Training 2026 - your name

Ingestion

Aggregations & products

Quality

Workspaces

Download data loader

+

CONNECTIONS

• GEOGLOWS Streamflow Forecasts

1 task · CSV

GEOGLOWS Streamflow Forecasts

CSV

1 task • 1 pending

Status filter

TASK NAME ↑ 1	STATUS	ACTIONS
Daily Nzoia GEOGLOWS Streamflow Forecast	LAST NEXT - 10 Sep 2026, 9:47 AM	Run now Details >

+ Add data connection

<https://playaround.hydroserver.org/orchestration>

The ETL Task has been successfully created and scheduled. On a HydroServer deployment with ETL orchestration enabled, the worker would automatically execute the task at the scheduled time.



[Browse monitoring sites](#)[Your sites](#)[Visualize data](#)[Data management](#)[About](#)

Job orchestration

Selected workspace: Kenya Training 2026 - your name

Ingestion

Aggregations & products

Quality

Workspaces

Download data loader

CONNECTIONS

+

Search connections...

• GEOGLOWS Streamflow Forecasts

1 task · CSV

← GEOGLOWS Streamflow Forecasts

Daily Nzoia GEOGLOWS Streamflow Forecast

Pending

Every 1 days

Pause

Edit

Delete

Run now

Run history

Mappings

Pending

Run queued.

RUN 01A08718

STARTED

9 Sep 2026, 9:55 AM (America/Phoenix)

Queued

Pending

Run queued.

RUN 01A08718

STARTED

9 Sep 2026, 9:54 AM (America/Phoenix)

Queued

Pending

Run queued.

RUN 01A08717

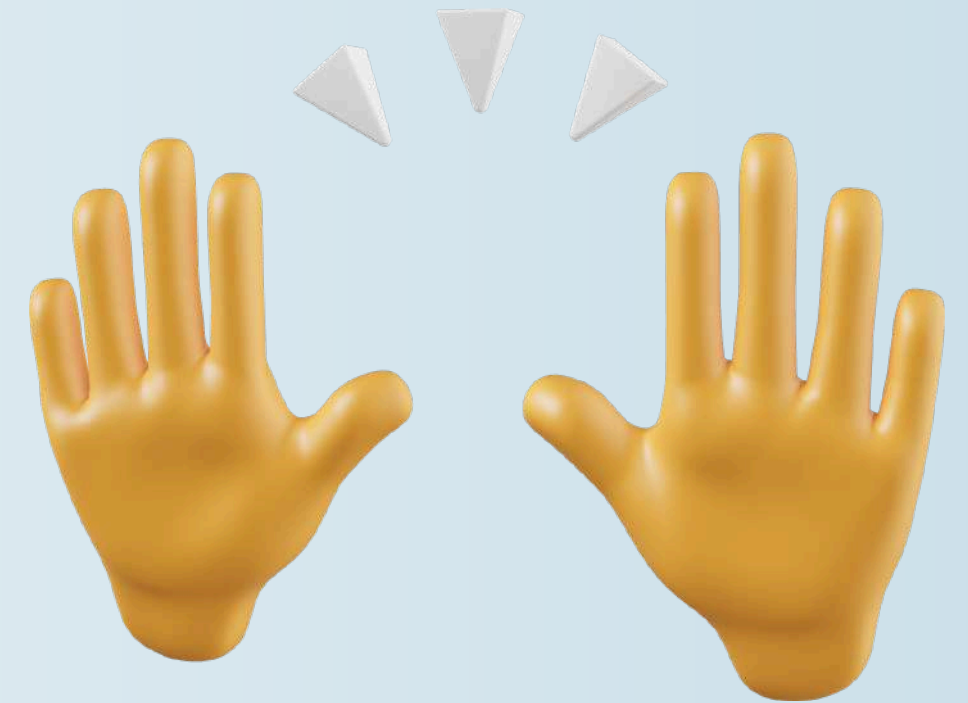
STARTED

9 Sep 2026, 9:54 AM (America/Phoenix)

Queued

+ Add data connection

Good job! 🎉 😎



**You have successfully explored
different methods for automating
hydrological data ingestion in
HydroServer!** 🎉 🐱

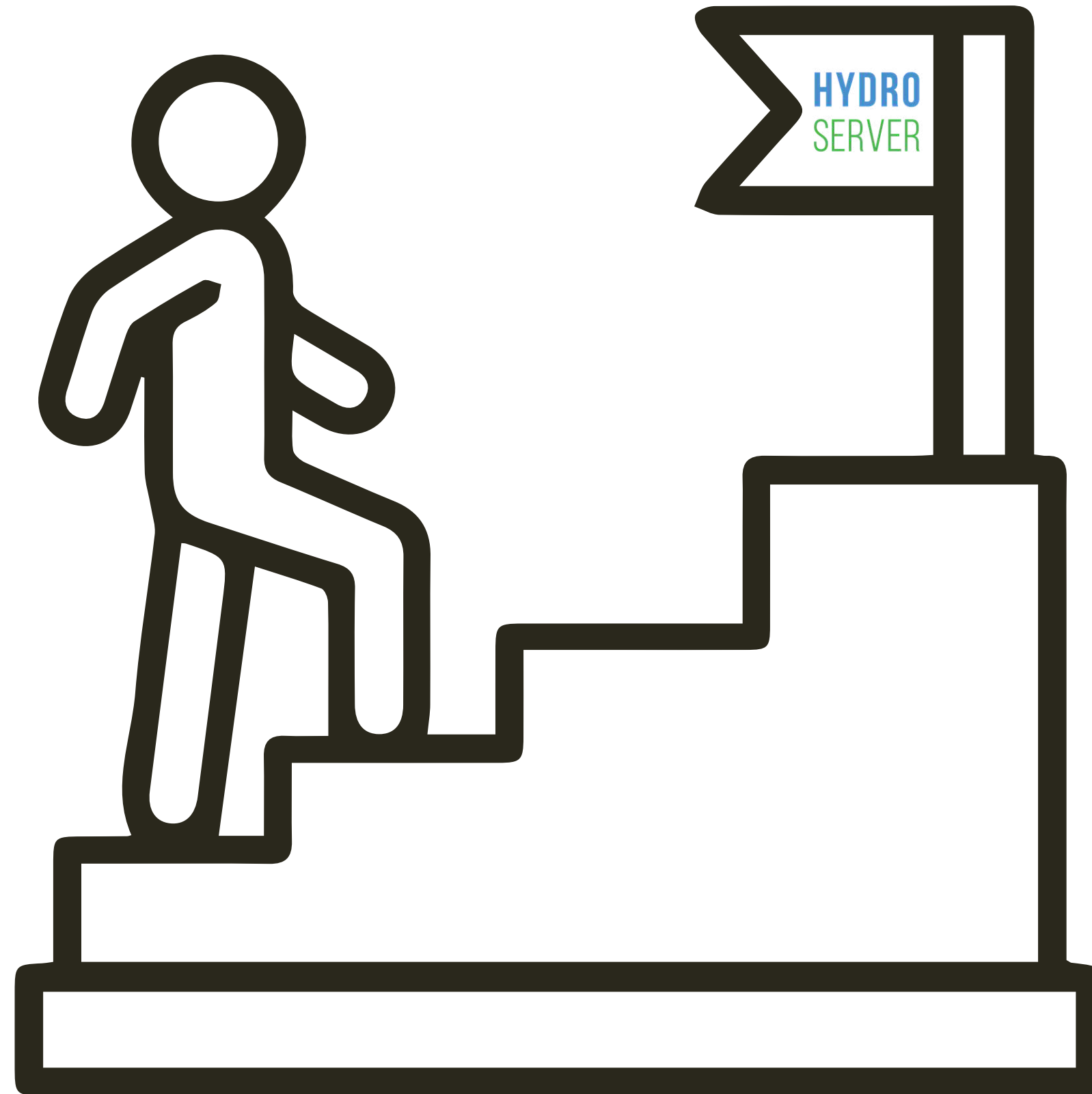


Wow, you've come a long way!

**and you've learned and shared so
much!**



that finally you have made it!



Congratulations!

Training finished!

